



iPASeIoT: An intelligent pipeline for automatic and adaptive feature extraction for secure IoT device identification and intrusion detection

Ogobuchi Daniel Okey ^a, Sajjad Dadkhah ^{b,*}, Heather Molyneaux ^c,
Demóstenes Zegarra Rodríguez ^d, João Henrique Kleinschmidt ^a

^a Center for Engineering, Modeling and Applied Social Sciences(CECS), Federal University of ABC, Santo André, 09580-280, São Paulo, Brazil

^b Canadian Institute for Cybersecurity, Faculty of Computer Science, University of New Brunswick, Fredericton, E3B 5A3, New Brunswick, Canada

^c National Research Council Canada (NRC), Fredericton, E3B 9W4, New Brunswick, Canada

^d Department of Computer Science, Federal University of Lavras, Lavras, 37200-000, Minas Gerais, Brazil

ARTICLE INFO

Keywords:

Device identification
Device fingerprinting
Adaptive feature extraction
Intrusion detection
Model reliability
Generalizable models
Scalable IDS framework

ABSTRACT

The widespread integration of Internet of Things (IoT) devices has enhanced the intelligence of homes, industries, and offices, yet it introduces critical security challenges due to their susceptibility to dynamic threats and behavioral heterogeneity, necessitating identification via communication patterns rather than mere physical recognition. This paper addresses the demand for a unified security framework in IoT ecosystems, where devices, limited by diverse protocols and constrained computational resources, face attacks such as DNS tunneling, MAC spoofing, and several other threats. Existing approaches, which rely on coarse-grained signatures or segregated machine learning for device identification and intrusion detection, exhibit limited resilience, increased operational overhead, poor cross-network adaptability, and scalability constraints in real-time dynamic settings. We propose iPASeIoT, a single-model framework that concurrently identifies IoT devices and detects intrusions using fine-grained behavioral fingerprints. Our methodology combines machine and deep learning algorithms with a modified firefly algorithm employing a kappa score-based voting mechanism for adaptive feature selection, yielding a lightweight, resource-efficient model by optimizing agreement beyond chance across network traffic, inter-arrival times, and protocol-specific features. Evaluated on the CICIoMT2024, CICIoT2023, and UNSW2019 datasets, iPASeIoT achieves mean F1 scores of 99.99%, 99.88%, and 98.35% for device identification and 99.96%, 99.38%, and 98.79% for threat classification across the CICIoMT2024, CICIoT2023, and UNSW2019 datasets, respectively. With a mean inference time of 0.0005 seconds per sample and a mean Hamming loss of ≈ 0.001 , iPASeIoT provides a pioneering, efficient, and scalable solution to counter evolving security threats in heterogeneous IoT environment.

* Corresponding author.

E-mail addresses: okey.ogobuchi@ufabc.edu.br (O.D. Okey), sdadkhah@unb.ca (S. Dadkhah), heather.molyneaux@nrc-cnrc.gc.ca (H. Molyneaux), demostenes.zegarra@ufpa.br (D.Z. Rodríguez), joao.kleinschmidt@ufabc.edu.br (J.H. Kleinschmidt).

URL: <https://orcid.org/0000-0003-0686-2763> (O.D. Okey), <https://orcid.org/0000-0002-5582-0255> (S. Dadkhah), <https://orcid.org/0000-0001-5401-7551> (D.Z. Rodríguez), <https://orcid.org/0000-0001-6706-455X> (J.H. Kleinschmidt)

<https://doi.org/10.1016/j.iot.2025.101802>

Received 18 June 2025; Received in revised form 12 September 2025; Accepted 16 October 2025

Available online 25 October 2025

2542-6605/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

1. Introduction

The Internet of Things (IoT) comprises physical objects embedded with sensors and software, enabling connectivity and data exchange across diverse environments. This vast and rapidly expanding network encompasses a wide range of devices, from consumer-grade smart home gadgets to highly sophisticated industrial sensors. According to a Statista report (2024), the number of connected devices is projected to exceed 40.1 billion by 2030 [1], making the IoT network one of the fastest-growing technologies recently. This surge is expected to drive market revenue to approximately US\$1.06 trillion by 2025 worldwide. Furthermore, with an annual growth rate of 10.17% (CAGR 2025–2029), the market volume is projected to reach US\$1.56 trillion by 2029, according to the Statista Report [1]. This projection aligns with the Global IoT Device Quantity Forecast Report released by IoT Analytics in 2023 [2], which forecasts the global number of IoT devices to reach over 29 billion by 2027, representing approximately a fivefold increase over the 6 billion devices connected in 2017 [3].

Dadkhah et al. [4] argue that IoT devices in smart cities, smart homes, healthcare, control and automation systems, and smart manufacturing provide convenience, efficiency, and productivity. Consequently, the security and management of IoT systems are crucial for ensuring their reliability and trustworthiness, especially as they gather, manage, store, and transmit critical information. As noted by He et al. [5], these devices transmit substantial volumes of data, making them prime targets for cyber threats [6]. Many companies manufacture IoT devices that prioritize practicality over security, leading to vulnerabilities and privacy concerns. Furthermore, many users lack the expertise in security required to protect their devices, creating an additional layer of vulnerability that can be easily exploited [7].

Recently, The Hacker News highlighted that numerous vulnerabilities are inherent in IoT devices. For example, Quarkslab, a well-known cybersecurity firm, discovered and reported a vulnerability known as TPM2.0 in November 2022, which could potentially affect billions of IoT devices [8]. Likewise, the NETSCOUT Threat Intelligence Report indicates that newly connected IoT devices are frequently targeted by malicious actors within five minutes of joining a network and typically become specific targets within 24 hours [9]. Most of these attacks exploit the inherent vulnerabilities in the devices themselves [10,11]. Compromised IoT devices are frequently attacked and often hijacked for use in botnets, such as Mirai [12], URSNIF, and BASHLITE, where these compromised devices were used to carry out large-scale Distributed Denial of Service (DDoS) attacks.

Therefore, IoT devices require effective management and security through external measures, as their limited energy and memory capacities preclude the use of robust computational encryption mechanisms [13]. One approach is for network administrators to be well informed about connected devices to easily pinpoint compromised ones. The Media Access Control (MAC) and Internet Protocol (IP) addresses are often used to distinguish connected devices. However, recent studies have shown that MAC addresses do not provide a secure method for identifying devices, as they can be easily disguised, impersonated, or altered [14]. Furthermore, different IoT devices use various protocols to communicate with vendor servers, complicating network management [15]. It is imperative to use distinct, non-guessable, and uncommon device attributes derived from communication patterns based on network traffic to distinguish connected devices. These unique attributes, or fingerprints, are the digital DNA of IoT devices that differentiate them from other devices on the network. The process of obtaining these digital imprints is referred to as fingerprinting.

Fingerprints, such as radio frequency waveforms or clock skews, are extracted from the physical layer, whereas flow patterns or packets are derived from device communication at the network layer. The sensitivity to environmental factors and the strict dependence on specific wireless technologies limit the applicability of physical-layer data [16]. In contrast, network traffic data offer a scalable, dynamic, and robust interface that is more complex to analyze. In this regard, recent studies [3,17–23] have focused on using device-specific attributes derived from network traffic to develop intelligent machine learning (ML) models for device identification. Unlike credential-based authentication, which is susceptible to spoofing or theft, device fingerprinting leverages network traffic patterns, such as packet inter-arrival times, protocol headers, or statistical aggregates, to establish a device's identity.

The significance of IoT device fingerprinting and identification is multifaceted. It improves network security by facilitating accurate device authentication, thereby reducing vulnerability to unauthorized access and cyber threats. Additionally, it streamlines device management by providing a reliable mechanism for tracking, monitoring, and updating devices in large-scale IoT ecosystems. Moreover, device identification safeguards data integrity by authenticating data sources, a critical requirement in domains such as healthcare and industrial automation, where precision is essential. Beyond these functional benefits, it also enables the delivery of personalized and context-aware services in smart environments by accurately identifying user devices.

Existing studies, including [18,19,22,24–27], have shown that distinct attributes derived from network packet headers can be effectively used to identify connected devices. These packet headers, when analyzed, provide several informative metrics that are instrumental in network-based tasks [17]. As shown in Fig. 1, a typical packet includes header components such as source and destination IP and MAC addresses, protocol type, Time-to-Live (TTL), flags, port numbers, sequence and acknowledgment identifiers, raw data, and checksums. This figure offers a clear view of the various attributes of a packet that can be leveraged to identify devices or classify packets.

While device identification (DI) identifies connected devices through fingerprints, intrusion detection systems (IDS) [28–31] are developed to detect and classify a wide range of network attacks, such as denial-of-service (DoS), botnet enlistment, protocol exploits, and man-in-the-middle (MitM). Some studies focus on either a binary classification task (classifying network packets as either benign or malicious) or a multiclass problem where multiple attack types are investigated. For example, Mani et al. [32] introduced a lightweight federated IDS employing hybrid feature selection and multiple supervised learning models, while Morales et al. [29] proposed a distributed network IDS that uses collaborative, federated ML, evaluated in an IoT testbed. Nguyen et al. [30] presented “Re-alguard,” a DNN-based network IDS optimized for IoT gateways. Ma et al. [33] developed ADCL, a collaborative detection framework

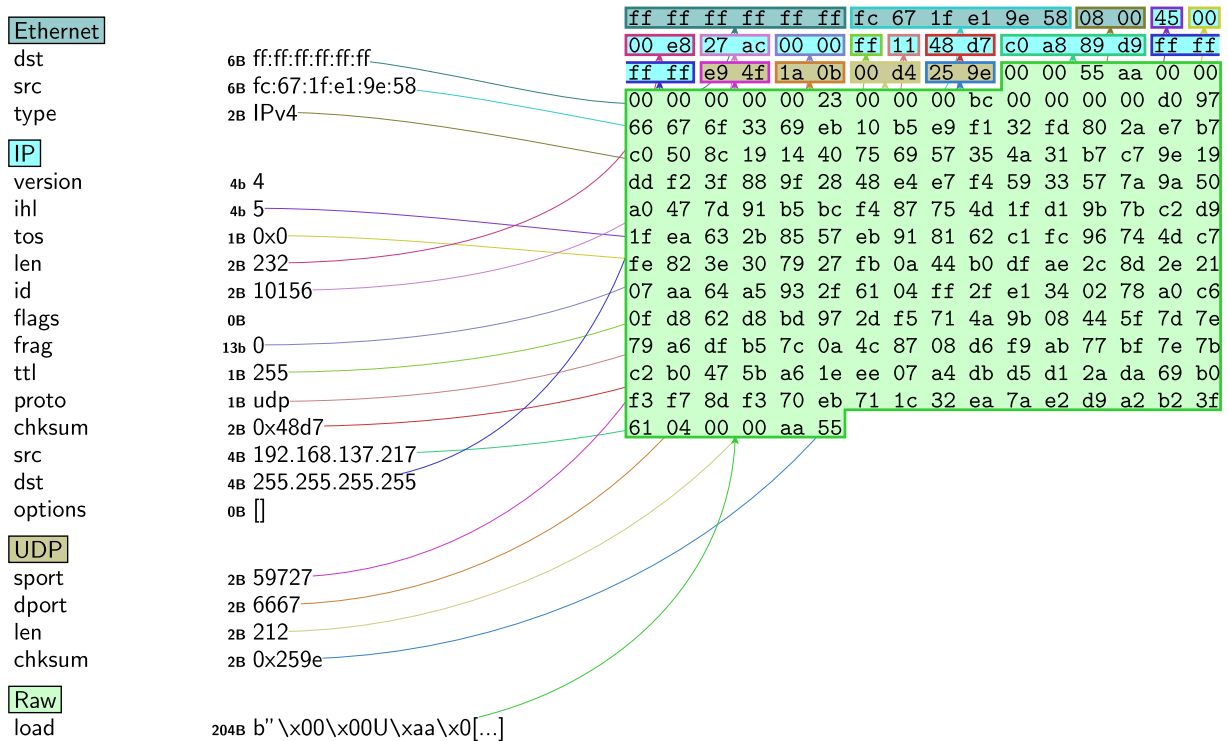


Fig. 1. Packet header and raw data information of the SOS Multifunctional Pager device in active state from the CICIoMT2024 dataset.

in which models trained in similar environments guide each other. Similarly, Okey et al. [28] presented an ensemble approach to IDS model design that leverages boosting classifiers to perform a multiclass classification task with an accuracy of up to 99.99%.

IoT security has extensively explored device identification (DI) and IDS as separate techniques to enhance the security of connected devices. Both DI and IDS rely on AI-enabled models trained on comprehensive datasets of network traffic. Although the independent application of these methods has shown efficiency, the high cost of training each model for a specific task, coupled with the limited data available for such tasks, highlights the need for a unified model. This model should be trained using a combination of device fingerprints and statistical flow information, enabling it to dynamically and adaptively leverage various attributes of network packets to perform both DI and IDS functions simultaneously. Furthermore, DI enhances network visibility, supports policy enforcement, and aids in detecting spoofed or unauthorized devices, thereby strengthening the overall security posture of the IoT network. In resource-constrained settings, it also facilitates lightweight and efficient security mechanisms aligned with the capabilities of individual devices. Thus, integrating device identification into IDS is fundamental for achieving scalable, intelligent, and resource-aware threat detection and improving IoT security. This area remains insufficiently explored in IoT security.

Consequently, this study proposes a robust, adaptive, and optimized method to identify IoT devices and detect intrusive network traffic using packet-level traffic, behavioral patterns, and statistical network attributes. By integrating behavioral and statistical properties, we create a scalable feature set that effectively accommodates both DI and IDS scenarios. We identify several key challenges in implementing this method: (1) The high complexity of feature extraction arises from the difficulty of deriving common features that generalize across diverse devices; (2) As more devices connect to the network, the model must relearn their attributes to effectively detect them, leading to performance degradation over time due to model drift; (3) Similarities in device design or devices produced by the same manufacturer create confusion as the model attempts to differentiate them; (4) With an increasing number of connections, the model's efficiency is likely to decline due to increased resource consumption, resulting in decreased model utility, as noted by Yin et al. [34].

We address these challenges by exploring the following questions: (1) To what extent can fine-grained, granular packet-level fingerprints bolster the robustness of IoT device identification models across heterogeneous network environments? (2) How do cross-device and cross-domain similarities in IoT packet signatures affect the specificity of device identification, and what effective mitigation strategies can be employed for overlapping features? We develop a multioutput classification model based on decision tree (DT), random forest (RF), XGBoost (XGB), and a 1-dimensional convolutional neural network (1D-CNN). The model employs a meta-learning approach, using DT, RF, and XGB as estimators to train a single model that yields multiple output prediction probabilities, representing both the device type and network traffic type. Furthermore, to address challenges (2) and (3), we introduce device- and threat-specific feature adaptation, enabling the model to retain crucial information about each device over time without requiring relearning. With these specific fingerprints, similar devices are readily identified without compromising precision. We optimize the initial 389 fingerprints extracted from individual packets using a lightweight Kappa-score-enhanced firefly optimization algorithm to reduce complexity issues.

Table 1
Comparison of IoT device identification methods.

Identification Method	Mode of Operation (Working Principle)	Strengths	Weaknesses	Application Examples
Port-based [17]	Identifies devices/services by analyzing open ports and protocols (e.g., HTTP on port 80, SSH on 22).	Benefits from fast classification, low computational cost, and easy implementation.	No deep inspection, limited to known services, and easily evaded (ports can be changed).	Applied in network scanners (Nmap) and basic IoT device configuration.
Statistical [21]	Uses traffic metrics (e.g., packet size, frequency, flow duration) to profile devices via ML or statistical methods.	No deep packet inspection required, harder to evade than port-based, and requires fewer computational resources for fingerprinting.	Tendency to have low accuracy for encrypted traffic and may require larger datasets.	Anomaly detection in smart homes and IoT botnet analysis.
Behavioral [17,19]	Profiles devices based on communication patterns (e.g., timing, request frequency, protocol sequences).	Resilient to encryption, adaptable to zero-day attacks, and capable of identifying unknown devices.	May have high false positives and typically requires large computational resources.	Identifying compromised IoT devices (e.g., cameras) and spoofed devices.
Payload-based [38]	Inspects packet contents (e.g., headers, payload data) for signatures or anomalies.	Highly accurate and can detect specific malware patterns.	Raises significant privacy concerns, high overhead (deep inspection), and fails when analyzing encrypted traffic.	Intrusion detection systems (e.g., Snort) and malware identification in firmware updates.

We outline the following contributions, which coalesce into a multifaceted framework that enhances existing research in IoT DI and introduces the integration of IDS as a significant advancement, with implications for network security, device management, and scalable IoT deployments.

1. A quantitative assessment of fine-grained packet-level, behavioral, and statistical fingerprints, illustrating their ability to bolster robustness across heterogeneous networks through a novel deep learning and domain adaptation framework.
2. An adaptive machine learning pipeline designed to address temporal changes via dynamic feature adjustments based on their detected importance and a reduced feature space through FOA-CKSV, as described in [Section 3.4.2](#).
3. An analysis of cross-device and cross-domain similarities, accompanied by mitigation strategies that utilize discriminative feature engineering and adaptive learning to enhance identification specificity.
4. A multioutput model that has robust capabilities to conduct both device identification and intrusion detection concurrently. To the best of our knowledge, this paper marks the first application of this approach to IoT security.
5. iPASecIoT, an optimized, all-in-one model that efficiently identifies devices and detects network traffic types through well-engineered network fingerprints, thereby reducing deployment and training costs.
6. Further tuning of iPASecIoT to adaptively select device- or traffic-specific features during inference, reducing computational complexity and inference time by up to 92.9937 % using the random forest classifier, as described in [Section 4.4](#).

The rest of the paper is organized as follows: [Section 2](#) highlights the existing literature and identifies gaps for our study. The proposed approach and implementation are discussed in [Section 3](#). In [Section 4](#), the results of the study are presented and discussed, while [Section 5](#) provides the conclusion of the study.

2. Related work

2.1. Theoretical background on IoT device fingerprinting & identification

IoT device identification models are categorized into active and passive approaches based on fingerprint acquisition [3]. Active methods involve sending probes to elicit device responses, revealing operating system, service, and configuration details for classification by type, manufacturer, or model [7]. These techniques excel in rapid, broad detection for asset discovery and network inventory but strain resources and may trigger security defenses due to intrusive probing.

Passive methods, in contrast, silently monitor regular network traffic to infer device characteristics, offering a discreet, lightweight solution ideal for intrusion detection, access control, and malicious activity identification [3]. The process includes capturing traffic at gateways, processing data, and classifying devices using packet capture (PCAP) files. These files store multi-layer features (data link, network, transport, application) to create unique traffic fingerprints, a method widely adopted in recent studies [11,17,35–37]. Similarly, iPASecIoT builds upon this ideology.

Early fingerprinting used easily spoofed coarse-grained features, such as MAC or IP addresses [18]. Modern approaches leverage fine-grained packet-level features, such as inter-arrival times (IAT), packet sizes, and protocol headers (e.g., TCP window sizes, MQTT client IDs), achieving up to 85 % accuracy in Wi-Fi networks, though performance varies with network conditions, such as Zigbee noise [19,21]. Feature extraction now spans flow-based (statistical traits), packet-based, and behavior-based (temporal patterns) domains [36], while recent studies extend it to include port-based, statistical, behavioral, and payload-based approaches, leveraging network packet components (headers, payloads, trailers) [3,7], as summarized in [Table 1](#).

Table 2

Comparative analysis of IoT device fingerprinting methods.

Method	Accuracy (%)	Task Type	Privacy-Oriented	No. of Devices	No. of Features
TinyDevID [36]	91 (max 97)	DI	–	3	77 (10 used) Flow-based
LFloTDI [22]	99.08 (CIC), 98.15 (UNSW), 95.28 (SMPS)	DI, DF	–	CIC: 38, SMPS: 13, UNSW: 18	53 (17 used) Packet, statistical
FICDF [37]	94	DF, DI	FL with Gaussian Noise	IoT Sentinel: 27, UNSW: 31	19 Binary packet
Legkodymov and Levshun [23]	99.4	DI, DF, Attack	–	CICIoT2022: 26	Original + 21 new Packet-level
Rabbani et al. [35]	99.95	DF, DI	–	CICIoT: 39, Aalto: 27, Combined: 66	47 (7–35 after FS) Per-packet, flow-based
Dora et al. [39]	RF: 81.36, SVM: 86.02, LR: 81.33	DI, DF	–	6 sensors + gateway	23 Header, statistical
IoTPROFILE [41]	–	DI	Low computational cost	UNSW: 27, UNB: 30	Packet length sequences Time-series dictionary
HFedDI [40]	–	DI	Horizontal FL	UNSW ACM: 10, Aalto: 16, UNSW TMC: 31	25 Flow-level (binary, integer)
Zhu and Lu [14]	–	DI	–	Not specified	Not specified Time-series protocol

Abbreviations: DI = Device Identification, DF = Device Fingerprinting, FS = Feature Selection, RF = Random Forest, SVM = Support Vector Machine, LR = Logistic Regression, FL = Federated Learning.

Notes: Missing values are denoted by “–” due to lack of reported data. Feature types and counts reflect post-selection numbers where applicable Table 2.

2.2. Behavioral-based and traffic analysis approaches

The varied profiles of IoT devices, such as active/idle, ON/OFF, interacting, or voice-activated (e.g., smart plugs), constitute their dynamic behavior on the network. Xu et al. [22] advanced packet-based methods, extracting 17 protocol-based features from a single packet, achieving 99.08 % accuracy on the CIC dataset using a pruned decision tree. Their LFloTDI framework, optimized for resource-constrained devices, integrates MQTT for real-time updates but faces scalability issues in distributed networks. Rabbani et al. [35] proposed a lightweight model using enhanced behavioral features, avoiding MAC addresses, and achieved 99.95 % accuracy on CICIoT2022 and Aalto IoT datasets with a decision tree. It excels in HTTPS-encrypted traffic via JA3, handshake metadata, and user-agent data, although continuous learning is needed for long-term adaptability. Dora et al. [39] developed a real-time method, extracting 23 TCP/UDP features through IP address scanning, port scanning, and ARP spoofing, achieving 96 % accuracy on the Aalto dataset, but struggles with inactive devices during scanning. Kumar et al. [27] proposed DI4IoT, a two-stage ML framework that first separates IoT from non-IoT traffic and then identifies device types using optimized flow-based features.

2.3. Lightweight and resource-constrained frameworks

Lightweight frameworks address IoT device constraints. Chaudhary et al. [36] introduced TinyDevID, a TinyML-based solution for microcontrollers, using flow-based fingerprints from devices such as Netatmo IoT cameras and TP-Link Kasa Smart Plugs. With an extra tree classifier and Neural Architecture Search, it achieves 91–97 % accuracy; however, latency, memory, and diverse network testing are needed. Similarly, Xu et al. [22] proposed LFloTDI, extracting 17 protocol-based features, achieving 99.08 % accuracy on the CIC dataset with 3.52–4.34 MB memory usage. Similar approaches include Shenoy et al. [40] and Ding et al. [37].

2.4. Robustness, specificity, and cross-device challenges

Zhu and Lu [14], Zhao et al. [41] argued that network heterogeneity and device similarities pose ongoing challenges in IoT. Likewise, Sivanathan et al. [21] noted a 20 % accuracy drop for Wi-Fi-trained models on Zigbee-compliant devices due to latency and packet loss sensitivity. Chen et al. [42] improved accuracy to 88 % across Wi-Fi and 5G via domain adaptation, but relied on coarse features, and Miettinen et al. [18] noted that low-cost devices (e.g., ESP32 chipsets) show overlapping signatures, reducing specificity to 70 %. Statistical fingerprints (e.g., IAT variance) offer robustness, while metadata (e.g., HTTP headers) provide specificity but falter in noisy conditions, as observed by Wang et al. [16]. Our work evaluates fine-grained fingerprints and domain adaptation to address these issues.

Collectively, these studies demonstrate significant advances in IoT DI and IDS, from behavioral analysis and lightweight frameworks to privacy-preserving federated learning and time-series methods. However, amidst the challenges of scalability in dynamic

networks, generalizability across diverse datasets, an evolving threat landscape, and computational efficiency on resource-constrained devices, there has been no focus on the simultaneous application of a single model that dynamically identifies devices and mitigates intrusions. Although Rabbani et al. [43] claimed that their work addressed DI and anomaly detection simultaneously, it used separate models for each task, increasing computational cost. Our work builds on these foundations and offers a novel solution that learns from optimized fingerprints to concurrently identify devices and detect intrusions via a single model. Through adaptive feature selection, iPASeIoT offers a scalable and privacy-preserving solution for real-world IoT deployments, effectively integrating DI and IDS in a unified model with multiple output layers to enhance accuracy, efficiency, and security. The summary of the key works reviewed in this study is presented in Table 2.

3. Methodology

3.1. Problem definition and analysis

The objective is to develop a single predictive model, iPASeIoT, capable of simultaneously classifying the type of device and the type of network traffic from a given dataset. The dataset $\mathcal{D}$ consists of $\mathcal{N}$ samples, each with $\mathcal{M}$ input features and two multiclass target variables: device type (with C_d classes) and network traffic type (with C_t classes). The features are optimized using FOA-CKSV described in Section 3.4, which selects or transforms the input features to enhance predictive performance. The model is trained to minimize a joint loss function suitable for multiclass and multioutput classification, leveraging a multioutput channel architecture.

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i^d, y_i^t)\}_{i=1}^{\mathcal{N}}$ represent the dataset of $\mathcal{N}$ samples, where: $\mathbf{x}_i \in \mathbb{R}^{\mathcal{M}}$ is the feature vector for the i -th sample comprising fine-grained traffic attributes (e.g., packet sizes, inter-arrival times, and protocol headers). Then, $y_i^d \in \{1, \dots, C_d\}$ is the device-type label, with C_d being the number of device types, and $y_i^t \in \{1, \dots, C_t\}$ is the traffic classification label. FOA-CKSV transforms the original feature space $\mathbb{R}^{\mathcal{M}}$ into an optimized feature space $\mathbb{R}^{\mathcal{M}'}$, where $\mathcal{M}' \leq \mathcal{M}$ and can be defined according to Eq. 1.

$$\mathbf{x}_i' = \phi(\mathbf{x}_i; \theta_{\text{FOA-CKSV}}), \quad (1)$$

where $\phi: \mathbb{R}^{\mathcal{M}} \rightarrow \mathbb{R}^{\mathcal{M}'}$ is the feature optimization function parameterized by $\theta_{\text{FOA-CKSV}}$, determined through Algorithm 1, and the optimized or reduced dataset becomes $\mathcal{D}' = \{(\mathbf{x}_i', y_i^d, y_i^t)\}_{i=1}^{\mathcal{N}}$.

Therefore, the single model, iPASeIoT $f_\theta: \mathbb{R}^{\mathcal{M}} \rightarrow \mathbb{R}^{C_d} \times \mathbb{R}^{C_t}$ maps $\mathbf{x}_i$ to a tuple (p_i^d, p_i^t) , where: $p_i^d \in \mathbb{R}^{C_d}$ is the probability distribution over device types, and $p_i^t \in \mathbb{R}^{C_t}$ is the probability distribution over traffic types. The model f_θ is trained to minimize a combined loss function over both outputs specific to the classifier. In general, for a given traffic sample, iPASeIoT produces probabilities for all classes in two categories, corresponding to device identification or intrusion detection. For example, with the UNSW2019 dataset, which has benign and malicious samples (binary classification) for intrusion detection and 17 classes for device identification, different probabilities are generated for IDS and DI. The highest probability for DI indicates the actual class, and the same applies to IDS. This process occurs concurrently.

3.2. System model and design approach

Developing a scalable, adaptive, and generalizable model, such as iPASeIoT, requires a detailed strategy and layout, as illustrated in Fig. 2. Following the accurate selection of the required data, various feature extraction methods are studied to identify the best approach to extract fine-grained and robust fingerprints. The best method is applied to generate a corpus of fingerprints for each device, which are subsequently analyzed and optimized to select the most informative features. Furthermore, model optimization is performed to determine the best parameters that result in faster training and inference to mitigate resource unavailability issues, and finally, the trained models are evaluated across various data streams, according to Fig. 3. Generally, Fig. 3 is composed of six stages, including data gathering, dynamic feature extraction, dynamic data processing, adaptive feature selection, model selection, and evaluation. Each phase is characterized by a rigorous process that ensures that iPASeIoT is effectively developed.

3.3. Data acquisition and analysis

A high-performance DI and IDS model depends on network traffic data collected from diverse IoT devices in real-world or simulated scenarios. Gathering data from numerous devices often requires extended observation periods, making the process time-consuming and resource-intensive due to the inherent diversity of IoT systems. To address this, existing studies, such as those by Xu et al. [22], Rabbani et al. [35], Chaudhary et al. [36], use publicly available datasets, enhancing reproducibility and transparency.

To develop iPASeIoT, three datasets are used, including CICIoMT2024 by Dadkhah et al. [4], CICIoT2023 by Neto et al. [44], and UNSW2019 by Hamza et al. [45], to ensure robust DI and IDS in the domains of healthcare, home automation, and smart cities. CICIoMT2024, a recent release from the Canadian Institute for Cybersecurity (CIC), provides traffic from healthcare IoT devices (e.g., baby monitors, oxygen devices) over extended periods. Dadkhah et al. [4] provides the raw packet capture (PCAP) files for the dataset, organized according to the devices from which they are collected, thus providing 17 types of network threats (malicious and benign) and 40 devices covering real and simulated devices. The CICIoT2023 includes traffic from more than 60 real IoT devices from which 33 different types of traffic (malicious and benign) are collected. These traffic types are grouped into seven categories (e.g., DoS, DDoS, Web Attack). Both CICIoMT2024 and CICIoT2023 support various communication protocols, including WiFi, MQTT, and Bluetooth profiles in addition to common protocols for network communication.

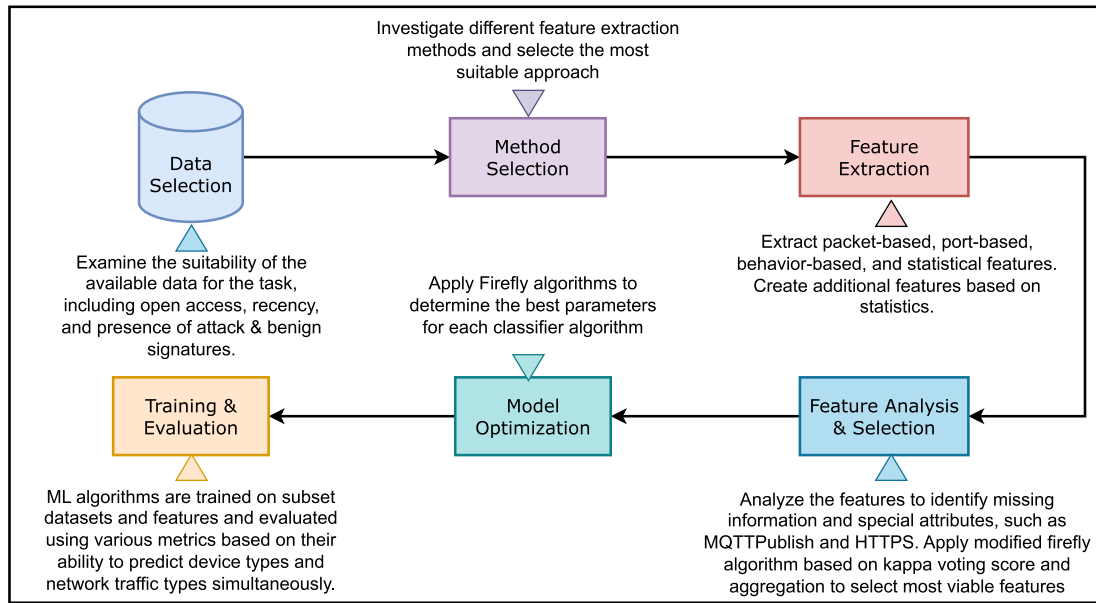


Fig. 2. General implementation steps for the proposed iPASecIoT.

Table 3

Distribution of the number of devices affected by each type of attack (CICIoT2023 dataset). While most of the attacks affected over 20 devices, the DoS-SYN and Reco-PortScan were found to attack only 7 and 9 devices, respectively.

Flow type	Number of Devices	Flow type	Number of Devices
MITM-ARPS	38	Vul-Scan	38
DDoS-HTTP	37	DDoS-SlowLoris	35
DNS-SPF	39	Benign	35
Mirai-UdpPlain	32	Recon-OSScan	22
DoS-HTTP	38	DDoS-ACKF	37
DDoS-ICMP	37	DDoS-PSHACK	38
DoS-UDPF	38	DDoS-UDPF	38
DDoS-ICMP-Frag	38	DDoS-TCPF	38
DDoS-SYN	37	mirai-Greip	31
DoS-TCPF	39	DDoS-UDPFra	36
DoS-SYN	7	DDoS-RSTFIN	37
DDoS-SYNNoIP	35	Mirai-Greet	33
Bhijacking	36	SQLInjection	37
Backdoor-Mal	32	XSS	31
Uploading	30	Recon-PingSweep	37
Recon-PortScan	9	CMDInfection	31
DcBruteForce	39	Recon-HD	36

Similarly, the UNSW ACM SOSR 2019 Hamza et al. [45] (referred to as UNSW2019, henceforth) contains attack and benign traces collected over 44 days in 2018, from 28 devices. During this period, benign traffic was captured for 27 days, and another 17 days featured a collection of packets that were both benign and malicious in origin. Unlike prior studies [20,22,36] that relied solely on benign traffic for DI, this work evaluates fingerprint generalizability across DI and IDS scenarios. When a device is under attack, its behavior is altered, creating a deviation from the existing fingerprints. Therefore, training an identification model on only benign traffic has limitations in terms of scalability and generalizability. Thus, we investigate how malicious packets alter behavioral patterns, potentially skewing detection models. By learning the attributes of a device from both malicious and normal traffic, the model becomes more robust to various device behaviors under varying conditions.

The three datasets are labeled for a multioutput scenario, containing devices and network traffic types (benign and different classes of malicious flows) as labels. All three datasets used, except CICIoT2023, have the PCAP files arranged according to devices and attacks, thus creating the labels was done by mapping the MAC address of individual packet with the MAC address provided along with the PCAP following the steps taken by Kostas et al. [17], Hamza et al. [45]. However, the CICIoT2023 PCAP files are arranged according to traffic patterns without a clear device distinction. Hence, there is a need to obtain device-specific packets from the original PCAP files. To achieve this, we group the packets by MAC addresses as provided by Neto et al. [44], creating new traffic packets for each in benign and malicious scenarios using the procedure depicted in Figure4. According to Figure4, for a SYN-Flood

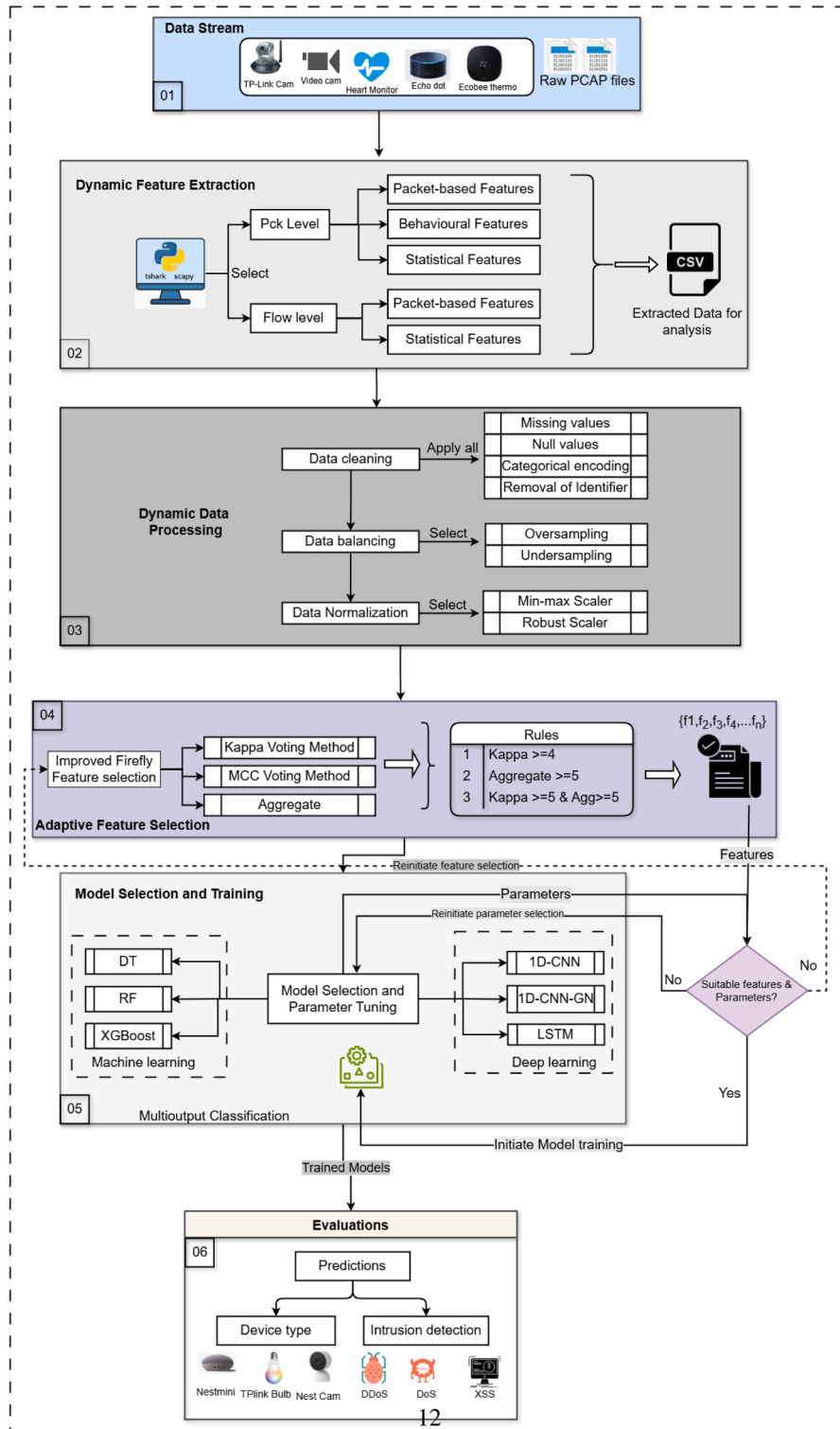


Fig. 3. Framework for implementing the proposed intelligent secure IoT system, detailing the data collection, processing methods, feature extraction using Python and tshark, dynamic feature optimization methods, model training algorithms, and evaluation.

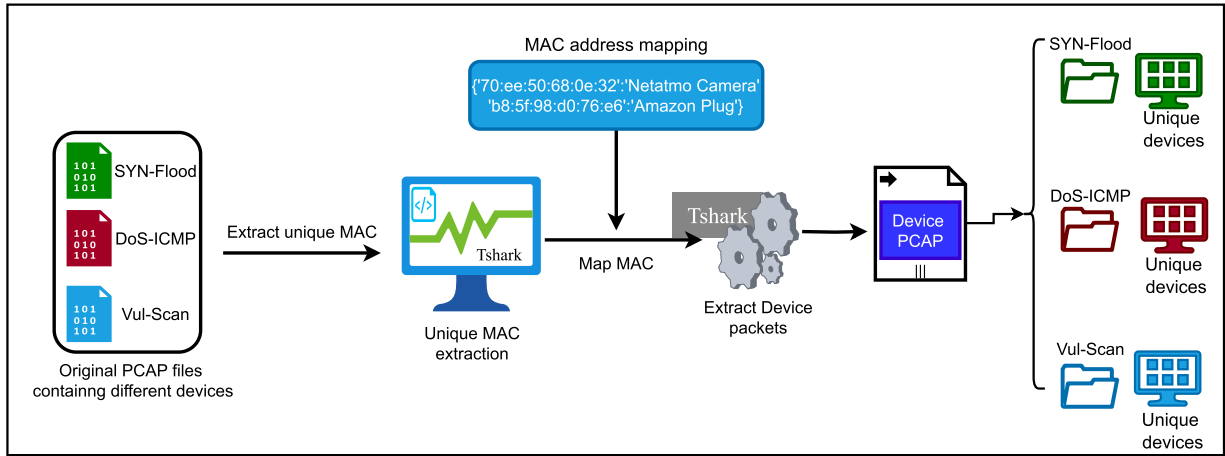


Fig. 4. Process for generating the device-specific PCAP file from the original network raw data. The original raw data were organized according to the network attacks. For each traffic, we generate packets specific to each device.

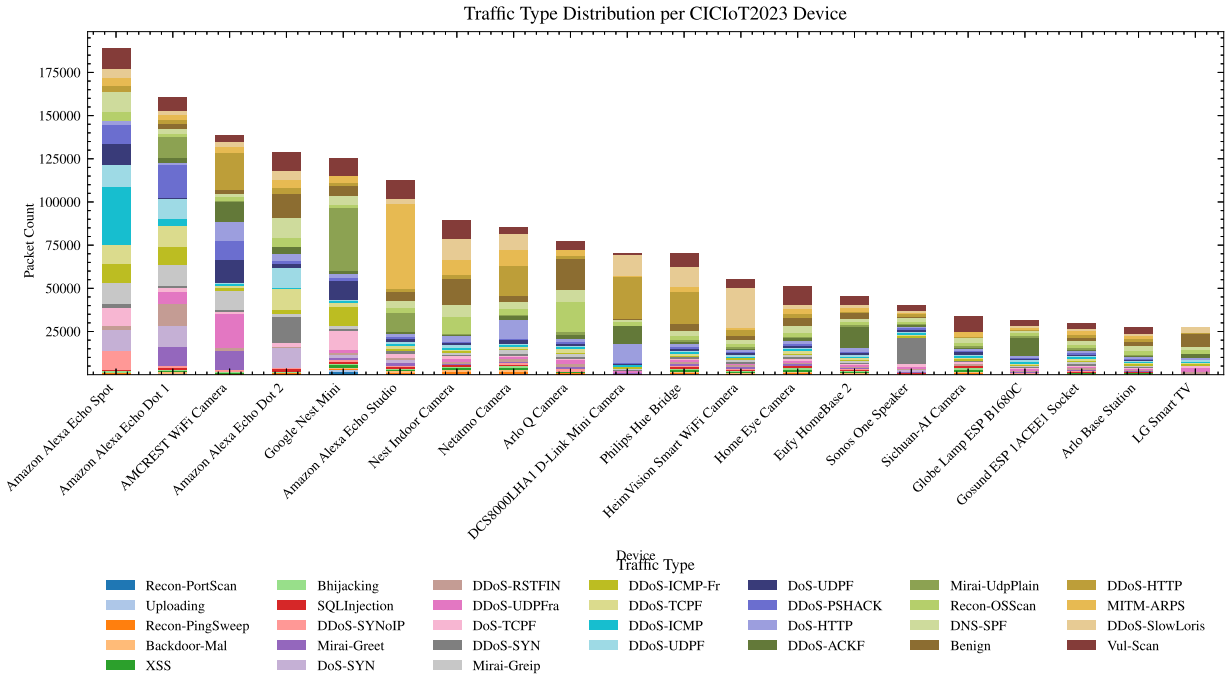


Fig. 5. Stacked distribution of samples in the CICIOT2023 dataset showing the various attacks and devices.

PCAP file comprising about 1.9 GB of network traffic, we extract all packets that belong to a unique MAC address, which are mapped to the MAC addresses provided by Neto et al. [44], resulting in separate packets for each device. This information is then used to create the labels for the network and device. Due to the large volume of data, we used the first 1.9 GB of each PCAP file. For each attack and benign category, we extracted devices based on either the source (eth.src) or target (eth.dst) MAC addresses. Table 3 summarizes the total number of devices affected by each attack. The task of classifying devices according to their types and detecting network traffic through these devices presents a novel and underexplored area in IoT security, a gap which this work bridges. A summary of extracted packets and devices is presented in Figure 5.

3.4. Feature extraction, engineering, and selection

Feature extraction methods in the literature include individual packet-based approaches, as used by Kostas et al. [17], Xu et al. [22], Kumar et al. [27], flow-based methods applied by Chaudhary et al. [36], Shenoy et al. [40], Rabbani et al. [43], and window-sizing approaches implemented by Charyyev and Gunes [46,47]. The individual packet-based method is widely adopted due to its robust features, as noted in [35,48]. Accordingly, we adopted this approach, extending feature extraction to include frequently used IoT protocols not extensively covered in prior studies, such as Message Queuing Telemetry Transport (MQTT), Real-Time Streaming

Protocol (RTSP), Session Traversal Utilities for NAT (STUN), and Hypertext Transfer Protocol Secure (HTTPS), with a deeper analysis of each feature's component attributes. Table 4 presents a summary of the extracted features, while full details of all features are available at <https://github.com/trimechnologies/FedULearn-IoT>. The extracted features cover four aspects: statistical, behavioral, protocol-based, and packet-level information from each network capture. Packet-level features describe properties of individual packets, such as size, payload length, and timestamps, providing a granular view of network activity. Behavioral features capture patterns of device or traffic behavior over time, including metrics like burst counts and port diversity, which reflect device interactions within a network. Protocol-based features are tied to specific network protocols, such as TCP, UDP, or TLS, capturing details like port numbers, protocol ratios, and TLS version information to highlight protocol-specific attributes. Statistical features, derived from computations like mean, variance, and skewness of packet sizes or inter-arrival times, provide insights into the distribution and variability of traffic, enabling the detection of anomalies or trends across multiple packets.

Data cleaning, encoding and engineering are conducted. In the extracted CSV files, we identified fields with missing or null values, which were critically analyzed to ensure that there is no bias in iPAsecIoT predictions. Some packet from the MQTT traffic contained publish and subscribe information, while some Domain Name Server (DNS) query data were identified for different packets. These categorical data were converted into 1s while missing values are dropped. In the feature selection stage, three approaches are adopted. In the first phase, we manually removed features that contained identifiable information, such as the ID, source IPs, destination IPs, source MAC and destination MAC among others. In the second phase, we applied variance analysis where we removed all features that have all zeros variance.

Lastly, the Firefly optimization algorithm (FOA) enhanced with the Cohen Kappa Score Voting (CKSV) mechanism, called FOA-CKSV, is applied to perform feature search with Decision Tree (DT) classifier. The DT is used to evaluate the performance of the selected individual feature as well as the total feature selected after a complete run using a defined kappa value as a threshold for selecting a particular feature.

3.4.1. Why feature selection (FS)?

Let us define the domain of the entire extracted fingerprint from individual packets, as F_N , according to Eq. (2), FS is aimed at selecting an optimal feature subset f' that best represents F_N such that a classifier algorithm learns from the data using f' in a similar way it would learn using F_N . We can represent the selected subset according to Eq. (3).

$$f = \{f_1, f_2, f_3, \dots, f_{F_N}\} \quad (2)$$

$$f' = \{f'_{n_1}, f'_{n_2}, f'_{n_3}, \dots, f'_{n_j}, \dots, f'_{n_F}\} \quad (3)$$

where F is the number of selected features and $f' \subseteq f$.

All three datasets used have the same feature space with sizes $F_N = 389$. Since a set of F_N features can generate 2^{F_N} distinct subsets, by a pairwise combination, the number of possible combinations of features becomes computationally expensive. Specifically, $2^{389} \approx 1.26 \times 10^{117}$ demonstrates the exponential complexity and computational infeasibility of exhaustive search methods. Random selection of features is unlikely to capture the underlying patterns or relationships in the data effectively. Therefore, the main objective of FS is to identify the most relevant and informative subset of features that can enhance model performance while reducing dimensionality. A well-selected subset minimizes redundancy and irrelevance, leading to better generalization and reduced overfitting.

3.4.2. Enhanced firefly optimization algorithm, FOA-CKSV

The FOA, initially proposed by Yang [49] in 2008, is a nature-inspired meta-heuristic algorithm that mimics the behavior of fireflies, where each firefly represents a potential solution x_i , leveraging the flashing behavior of fireflies, where brighter fireflies attract others. The algorithm optimizes the feature subset by maximizing a fitness function based on classification performance. FOA benefits from fast convergence [50], and therefore has been applied to select features for different tasks, including intrusion detection [51–53], informing our choice of algorithm. The enhanced FOA-CKSV applies DT with 10-fold cross-validation, with the performance measured through the ability of the selected features to yield a high Cohen's kappa score. During the search, the flies identify each feature with high brightness, attracting the search towards the optimal solution.

FOA establishes the following conditions in the optimization space: (1) **Attraction**: All fireflies are unisex, and a firefly is attracted to others based on their brightness (fitness). (2) **Brightness**: The brightness of a firefly is determined by the objective function. For FS, brightness corresponds to the quality of the feature subset based on kappa score. (3) **Distance-Based Attraction**: The attractiveness decreases with the distance between the fireflies, modeled using a light absorption coefficient. The following principles of FOA are applied in this work.

- **Firefly Representation**: Each firefly i is a binary vector x_i represented by Eq. (4), where $x_{ij} \in \{0, 1\}$ indicates whether a feature j is selected (1) or not (0). The dimensionality of the vector equals the number of features M .

$$x_i = [x_{i1}, x_{i2}, \dots, x_{iM}] \quad (4)$$

- **Fitness Function**: The fitness of a firefly is evaluated using a wrapper approach, where a DT classifier is trained on the selected feature subset, and the fitness is a weighted combination of kappa score and feature subset size as in Eq. (5).

$$f(x_i) = w_1 \cdot \text{Kappa}(x_i) - w_2 \cdot \frac{\sum_{j=1}^M x_{ij}}{M} \quad (5)$$

Here, w_1 and w_2 are weights (e.g., $w_1 = 0.9, w_2 = 0.1$) to balance kappa score and sparsity.

Table 4
General packet metadata features (all datasets).

Feature Name	Description	Data Type	Feature Type
Pck_size	Size of the packet in bytes.	int	Packet-level
Timestamp	Human-readable timestamp (YYYY-MM-DD HH:MM:SS.sssss).	str	Packet-level
Payload_bytes	Length of the packet payload in bytes.	int	Packet-level
Entropy	Shannon entropy of the packet payload.	float	Packet-level
IAT	Inter-arrival time between consecutive packets.	float	Packet-level
Burst_count	Number of packets in a burst (IAT < 0.1s).	int	Behavioral
IAT_mean	Mean inter-arrival time for packets.	float	Statistical
IAT_std	Standard deviation of inter-arrival times.	float	Statistical
Packet_freq	Packet frequency (1/IAT if IAT > 0).	float	Statistical
Direction_ratio	Ratio of outgoing to total packets.	float	Behavioral
Time_Since_Prev_Frame	Time since the previous packet in the capture.	float	Packet-level
Jitter	Difference between current IAT and mean IAT.	float	Statistical
Min_Elapsed_Time	Minimum elapsed time since the first packet.	float	Packet-level
Pck_Size_Sum	Sum of packet sizes.	int	Statistical
Pck_Size_Min	Minimum packet size observed.	int	Statistical
Pck_Size_Max	Maximum packet size observed.	int	Statistical
Pck_Size_Med	Median packet size observed.	int	Statistical
Pck_Size_Avg	Average packet size observed.	float	Statistical
Pck_Size_Var	Variance of packet sizes.	float	Statistical
Pck_Size_Q1	First quartile (25th percentile) of packet sizes.	int	Statistical
Pck_Size_Q3	Third quartile (75th percentile) of packet sizes.	int	Statistical
Pck_Size_IQR	Interquartile range (Q3 - Q1) of packet sizes.	int	Statistical
Pck_Size_Skew	Skewness of packet size distribution.	float	Statistical
Pck_Size_Kurt	Kurtosis of packet size distribution.	float	Statistical
Entropy_Min	Minimum entropy value observed.	float	Statistical
Entropy_Q1	First quartile of entropy values.	float	Statistical
Entropy_Var	Variance of entropy values.	float	Statistical
Payload_Len_Q1	First quartile of payload lengths.	int	Statistical
Payload_Len_Q3	Third quartile of payload lengths.	int	Statistical
Payload_Len_IQR	Interquartile range of payload lengths.	int	Statistical
Port_Diversity	Number of unique source and destination ports used.	int	Behavioral
Burst_Duration_Mean	Mean duration of packet bursts.	float	Statistical
Burst_Duration_Var	Variance of packet burst durations.	float	Statistical
Hourly_Activity	Normalized packet counts per hour of the day (comma-separated).	str	Behavioral
Most_Freq_Spot	Most frequent source and destination ports (format: sport:X,dport:Y).	str	Behavioral
Label	Device label from MAC mapping (or 'unknown').	str	Behavioral
MAC	Source MAC address of the packet.	str	Packet-level
Protocol_IP	IP protocol number (e.g., 6 for TCP, 17 for UDP).	int	Protocol-based
Protocol_TCP	Indicator of TCP protocol usage (1 if TCP).	int	Protocol-based
Protocol_UDP	Indicator of UDP protocol usage (1 if UDP).	int	Protocol-based
Protocol_TCP_Ratio	Ratio of TCP packets to total packets for the device.	float	Protocol-based
TLS_version	TLS version from ClientHello (e.g., 0x0303 for TLS 1.2).	str	Protocol-based
TLS_cipher_suites	Comma-separated list of cipher suites in ClientHello.	str	Protocol-based
TLS_ja3	JA3 fingerprint (version,ciphers,extensions).	str	Protocol-based
HTTP_cert	Indicator of TLS certificate presence (1 if present).	int	Protocol-based
HTTPS	Indicator of HTTPS usage (1 if port 443).	int	Protocol-based
Ether_type	Ethernet frame type (e.g., 0x0800 for IPv4, 0x86DD for IPv6).	int	Protocol-based
Ether_src	Source MAC address of the Ethernet frame.	str	Packet-level
Ether_dst	Destination MAC address of the Ethernet frame.	str	Packet-level
LLC_dsap	Destination Service Access Point in the LLC header.	int	Protocol-based
TCP_sport	TCP source port.	int	Protocol-based
TCP_dport	TCP destination port.	int	Protocol-based
TCP_mss_values	Maximum Segment Size value from TCP options.	int	Protocol-based
TCP_window_scaling	Window scaling factor from TCP options.	int	Protocol-based
Portcl_src	Source port class (based on predefined port list or ranges).	int	Protocol-based
Portcl_dst	Destination port class (based on predefined port list or ranges).	int	Protocol-based
DNS_tc	Truncation flag (1 if set).	int	Protocol-based
DNS_rd	Recursion Desired flag (1 if set).	int	Protocol-based
MDNS	Indicator of Multicast DNS usage (1 if port 5353).	int	Protocol-based

- **Distance:** The distance between two fireflies i and j is the Hamming distance between their feature vectors as described in Eq. (6). This metric counts the number of positions at which the corresponding binary values differ, effectively quantifying the dissimilarity in feature selection between two solutions.

$$r_{ij} = \sum_{k=1}^M |x_{ik} - x_{jk}| \quad (6)$$

Algorithm 1 Voting optimized firefly optimization for feature selection.

```

1: Input:
2:   Dataset  $D$  with  $M$  features, Number of fireflies  $N$ , Maximum iterations  $T$ , Attractiveness parameters:  $\beta_0, \gamma$ , Randomiza-
   tion parameter:  $\alpha$ , Number of runs  $N_{\text{runs}}$ , Voting threshold  $\theta$ , Fitness function  $f$  (e.g., DT accuracy - feature count penalty)
3: Output:
4:   Selected feature subset  $S$ 
5: Initialize:
6:   feature_counts  $\leftarrow$  zeros( $M$ ) ▷ Track feature selection frequency
7:   best_subsets  $\leftarrow$  [] ▷ Store optimal subsets from each run
8: for run = 1 to  $N_{\text{runs}}$  do
9:   Initialize  $N$  fireflies with random binary vectors  $\mathbf{x}_i \in \{0, 1\}^M$ 
10:  Evaluate fitness  $f(\mathbf{x}_i)$  for each firefly using DT classifier
11:  for  $t = 1$  to  $T$  do
12:    for each firefly  $i$  do
13:      for each firefly  $j$  do
14:        if  $f(\mathbf{x}_j) > f(\mathbf{x}_i)$  then
15:          Compute  $r_{ij} \leftarrow \text{Hamming}(\mathbf{x}_i, \mathbf{x}_j)$ 
16:          Compute  $\beta \leftarrow \beta_0 \exp(-\gamma r_{ij}^2)$ 
17:          Update Eq. (8) random( $-0.5, 0.5$ )
18:          Discretize  $\mathbf{x}_i$  using sigmoid function
19:          Evaluate new fitness  $f(\mathbf{x}_i)$ 
20:        end if
21:      end for
22:      if no firefly  $j$  has  $f(\mathbf{x}_j) > f(\mathbf{x}_i)$  then
23:        Update  $\mathbf{x}_i \leftarrow \mathbf{x}_i + \alpha \cdot \text{random}(-0.5, 0.5)$ 
24:        Discretize  $\mathbf{x}_i$ 
25:        Evaluate new fitness  $f(\mathbf{x}_i)$ 
26:      end if
27:    end for
28:  end for
29:  Select best firefly  $\mathbf{x}_{\text{best}}$  with highest  $f(\mathbf{x}_{\text{best}})$ 
30:  Append  $\mathbf{x}_{\text{best}}$  to best_subsets
31:  Update feature_counts  $\leftarrow$  feature_counts +  $\mathbf{x}_{\text{best}}$ 
32: end for
33: Voting:
34:   Initialize  $S \leftarrow$  []
35:   for each feature  $j = 1$  to  $M$  do
36:     if feature_counts[ $j$ ]  $\geq \theta \cdot N_{\text{runs}}$  then
37:       Add feature  $j$  to  $S$ 
38:     end if
39:   end for
40: Return  $S$ 

```

The RF is an ensemble of decision trees, each trained on random subsets of data and features, with predictions aggregated via majority voting. RF enhances robustness and mitigates overfitting compared to DT, making it suitable for diverse IoT traffic patterns, while XGB is an optimized gradient boosting framework that iteratively builds trees to minimize a loss function. XGB excels in multiclass tasks due to its scalability, regularization, and ability to handle imbalanced IoT traffic data, offering high accuracy and robustness. For these classical ML methods, we utilized the MultiOutputClassifier [54] from sklearn.multioutput to produce two probability outputs corresponding to device type and network traffic type. Each classifier is trained independently for each output, sharing the same input features.

On the other hand, the 1D-CNN is designed for sequential data, such as network traffic time series, extracting local patterns (e.g., packet sequences) through convolutional filters, followed by pooling and dense layers [31]. Our 1D-CNN architecture consists of two hidden layers with 64 and 128 neurons, respectively, and a 128-neuron dense output layer with two output heads to predict both labels simultaneously. The predictions are passed through a softmax function, optimized using the Adam optimizer, with loss evaluated via sparse categorical cross-entropy. This design enables the 1D-CNN to capture temporal and hierarchical features critical for DI and IDS.

The multiclass multioutput classification framework extends traditional classification by enabling simultaneous prediction of multiple discrete output variables, each with more than two classes. For example, in the context of the CICIoT2023 dataset, each network traffic instance represented by a high-dimensional feature vector $\mathbf{x}_i \in \mathbb{R}^d$ and the associated label vector $\mathbf{y}_i = (y_i^{(d)}, y_i^{(t)})$ comprises two

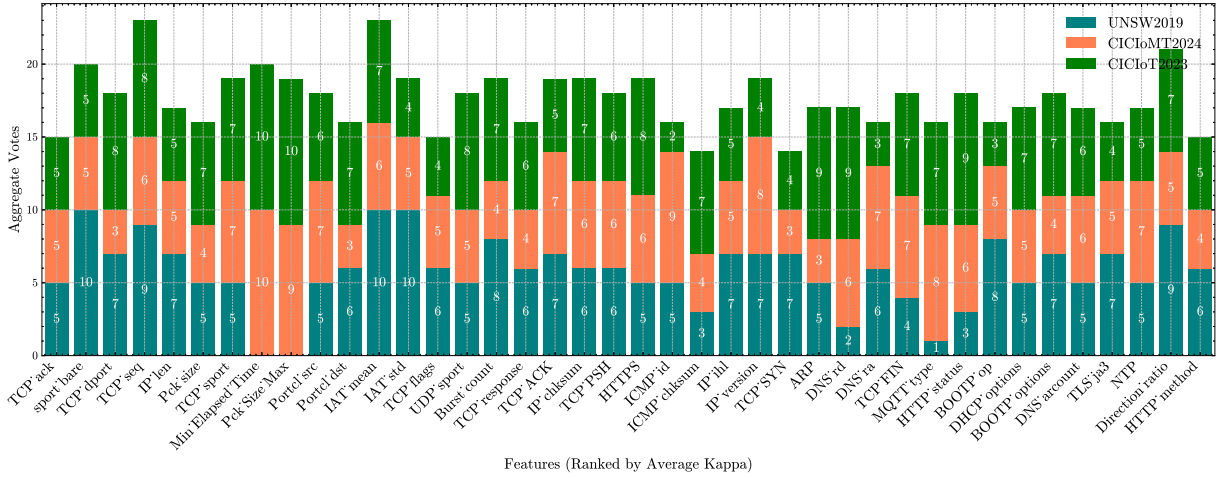


Fig. 7. Feature ranking based on kappa score and aggregated votes for the top 50 features of the datasets showing the number of aggregates per feature.

outputs: $y_i^{(t)}$ represents the attack category (e.g., DDoS-HTTP, Recon-Port, Benign), and $y_i^{(d)}$ denotes the device types (e.g., Echobee Camera, Google Nestmini, Alexa Echo Dot, TP Link Smart Plug). The learning objective is to jointly predict $(y_i^{(d)}, y_i^{(t)})$ given $\mathbf{x}_i$. For 1D-CNN, the features $\mathbf{x}_i$ are preprocessed into a structured representation and passed through a CNN-based feature extractor $\phi_\theta(\cdot)$. The extracted features are fed into two parallel softmax classification heads, one for each output. The forward pass is expressed as (11), where $\hat{\mathbf{p}}_i^{(k)}$ is the predicted probability distribution for the k -th output. The training minimizes the sum of categorical cross-entropy losses across the two outputs using Eq. 12, where $\aleph$ and ρ are task-specific weights balancing the contributions of each output.

$$\hat{\mathbf{p}}_i^{(k)} = \text{softmax}(W^{(k)} \phi_\theta(\mathbf{x}_i) + b^{(k)}), \quad k \in \{1, 2\} \quad (11)$$

$$\mathcal{L}_{\text{total}} = \aleph \cdot \mathcal{L}_{\text{CE}}(y^{(d)}, \hat{\mathbf{p}}^{(d)}) + \rho \cdot \mathcal{L}_{\text{CE}}(y^{(t)}, \hat{\mathbf{p}}^{(t)}) \quad (12)$$

Classical ML Algorithms use multioutput meta-estimators, training independent classifiers for each output variable while sharing the same input features. For DT, separate tree structures are learned for $y^{(t)}$ and $y^{(d)}$. For XGB and RF, independent ensembles of learners are trained per output, leveraging their respective strengths in handling complex decision boundaries. This approach offers several advantages for IoT DI and IDS including (1) Feature Sharing: A single feature extraction process supports both tasks, reducing computational redundancy and improving efficiency; (2) Predictive Consistency: the learning of one label influences the knowledge on the other, increasing reliability in the predictions; and (3) Scalability: The architecture is extensible to additional outputs without requiring significant redesign.

3.5.2. Description of the evaluation metrics used

Several metrics were evaluated to ensure that the model is robust, efficient and generalizes well on all datasets. Let TP, FP, TN, and FN represent the true positive, false positive, true negative and false negative respectively, the accuracy (Acc) measures the proportion of correct predictions, critical for overall DI and IDS performance according to Eq. (13).

$$Acc = \frac{TP + TN}{TP + FP + TN + FN} \quad (13)$$

The precision (Pre) evaluates correct positive predictions per class, vital for minimizing false alarms and calculated using Eq. (14).

$$Pre = \frac{TP}{TP + FP} \quad (14)$$

Recall (or sensitivity) gauges the fraction of actual positives identified, key for detecting attacks or rare devices applying the relation depicted in Eq. (15).

$$Rec = \frac{TP}{TP + FN} \quad (15)$$

F1 Score F1, also called the harmonic mean of precision and recall, balances both for robust evaluation according to Eq. (16).

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (16)$$

Since iPasecIoT handles a multioutput task, we introduce the Hamming Loss (Φ) that quantifies the fraction of incorrectly predicted labels in multioutput settings. If N is the number of samples, L is the number of labels, $y_{i,j}$ is the true label, $\hat{y}_{i,j}$ is the predicted label, and $\mathbb{I}$ is the indicator function (1 if true, 0 otherwise), crucial for joint DI and IDS accuracy, then, Φ can be expressed according to Eq. (17).

Table 5

Distribution of sample sizes across each dataset.

Dataset	No. of Devices	No. of Traffic Types
CICIoT2023	43	31
CICIoMT2024	17	16
UNSW2019	21	2
Combined	81	49

$$\Phi = \frac{1}{N \cdot L} \sum_{i=1}^N \sum_{j=1}^L \mathbb{I}(y_{i,j} \neq \hat{y}_{i,j}) \quad (17)$$

The Kappa Score assesses agreement between two raters or evaluators using $\text{Kappa} = \frac{p_o - p_e}{1 - p_e}$, where p_o is the observed Acc (proportion of correct predictions) and p_e is the expected Acc by random chance, accounting for class imbalance in multiclass tasks. The Matthews Correlation Coefficient defined by Eq. (18), provides a balanced measure for unbalanced data.

$$\text{MCC} = \frac{\text{TP} \cdot \text{TN} - \text{FP} \cdot \text{FN}}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}} \quad (18)$$

The Area Under the ROC Curve (AUC) that quantifies the trade-off between true positive rate ($TPR = \frac{\text{TP}}{\text{TP} + \text{FN}}$), false positive rate ($FPR = \frac{\text{FP}}{\text{FP} + \text{TN}}$), true negative rate, ($TNR = \frac{\text{TN}}{\text{TN} + \text{FP}}$) and (false negative rate ($FNR = \frac{\text{FN}}{\text{TP} + \text{FN}}$)) are evaluated to ensure reliability. These metrics collectively guide model optimisation, ensuring effective, efficient DI and IDS. Additionally, the *Train-Time* and *Test-Time* record durations for model training and prediction, crucial for resource-constrained IoT devices.

4. Results and discussion

This section presents the findings of our study, analyzing the performance of iPASecIoT across device identification (DI) and intrusion detection system (IDS) tasks. By analyzing the PCAP files for each dataset, we identify a broad range of prevalent and sophisticated threats targeting IoT devices. In both DI and IDS scenarios, the captured PCAP files consist of benign and attack traffic. During normal communication flows, we observe that the number of packets transmitted by a device is significantly lower compared to when the device is involved in an intrusion, either as a target or source. For example, a smart thermostat in the CICIoT2023 dataset typically generates fewer than 50 packets per minute during normal operation, primarily for periodic status updates via MQTT. In contrast, during a DDoS-HTTP attack, the same device may transmit over 500 packets per minute as it is exploited to flood a target server. Similarly, a baby monitor in the CICIoMT2024 dataset shows low packet rates (e.g., 20 packets per minute) for video streaming under benign conditions, but under a Recon-PortScan attack, packet rates spike to over 300 packets per minute due to aggressive port probing. In our evaluation, a total of 33 attack types are investigated in the CICIoT2023, 16 attacks in the CICIoMT2024, and UNSW2019 features a binary class (attack or benign). These intrusive attempts are categorized into seven types: DoS, DDoS, Mirai, Reconnaissance, Brute Force, Spoofing, and Web-based attacks. A summary of the number of instances is given in Table 5, where “No. of Traffic Types” denotes the number of threat classes in each dataset, including both malicious and benign traffic. Tables 6 and 7 provide a detailed summary of the datasets, including the number of samples for each specific class.

4.1. Performance analysis using all features selected with FOA-CKSV

This subsection presents the experimental results of iPASecIoT using features selected by the FOA-CKSV, highlighting its efficacy in simultaneous DI and IDS tasks. For CICIoT2023, 155 features were selected, 139 for UNSW2019, and 136 for CICIoMT2024. The 139 features selected for UNSW2019 was applied to each dataset and the result presented in Figs. 8 and 9. Each algorithm was trained on the dataset using a 70:30 split and evaluated with metrics defined in Section 3.5.2. Notably, FOA-CKSV identifies many common features across the three datasets, indicating similarities in device behaviors due to shared communication protocols and manufacturers. For example, the Amazon Echo in UNSW2019 shares similar characteristics with the Amazon Echo Spot and Echo Dot in CICIoT2023, including TCP Window Scaling, Entropy, TCP MSS values, packet size, IAT, HTTP method, and packet size mean/median. These commonalities underscore iPASecIoT’s ability to leverage robust, generalizable fingerprints, enhancing its applicability across diverse IoT environments.

Fig. 8 shows the kappa scores achieved by iPASecIoT, demonstrating its high accuracy in classifying both devices and network traffic using a reduced feature space. XGB-IDS achieves near-perfect kappa scores (0.9995-0.9997) across all datasets, reflecting exceptional performance in intrusion detection. Similarly, XGB-DI records kappa scores of 0.9895-0.9999, excelling in distinguishing device fingerprints, particularly on CICIoMT2024’s structured feature space. RF models (0.9863-0.9998) and DT models (0.9744-1.0) perform robustly but show slightly less consistency on CICIoT2023. The 1D-CNN-DI model records a kappa score of 0.8426 on CICIoT2023, indicating sensitivity to overlapping device behaviors, but improves significantly on CICIoMT2024 (0.9988). Training the 1D-CNN for only 10 epochs suggests potential for further improvement with extended training.

Fig. 9 presents an analysis of False Positive Rate (FPR) and Hamming Loss (Φ). XGB-IDS records near-zero FPR (0.0000-0.0052) on UNSW2019, minimizing false alarms critical for practical IDS deployment. RF-IDS and DT-IDS maintain low FPRs (0.0000-0.0063),

Table 6

Distribution of devices and traffic types across CICIoT2023 dataset, detailing sample counts for each device and traffic category (benign and attack).

Dataset	Device Distribution		Traffic Distribution	
	Device	Samples	Traffic Type	Samples
CICIoT2023	Amazon Echo Spot	189,160	Vul-Scan	72,518
	Amazon Echo Dot 1	160,527	MITM-ARPS	55,748
	AMCREST Camera	138,834	DDoS-SlowLoris	55,560
	Amazon Echo Dot 2	128,957	DDoS-HTTP	50,688
	Google Nest Mini	125,072	Benign	49,010
	Echo Studio	112,849	DNS-SPF	42,301
	Nest Indoor Camera	89,617	Recon-OSScan	37,879
	Netatmo Camera	85,384	Mirai-UdpPlain	33,141
	Arlo Q Camera	77,068	DDoS-ACKF	32,523
	DCS8000LHA1 Mini Cam	70,487	DoS-HTTP	31,519
	Philips Hue Bridge	70,234	DDoS-PSHACK	27,925
	HeimVision Camera	55,179	DDoS-ICMP	26,723
	Home Eye Camera	51,155	DDoS-UDPF	26,715
	Eufy HomeBase 2	45,615	DoS-UDPF	26,685
	Sonos One Speaker	39,937	DDoS-TCPF	26,470
	Sichuan-AI Camera	33,922	DDoS-ICMP-Frag	25,940
	Globe Lamp ESP	31,160	mirai-Greip	24,790
	Gosund Socket 1	29,677	DDoS-SYN	24,612
	Arlo Base Station	27,682	DDoS-UDPFra	22,482
	LG Smart TV	27,581	DoS-TCPF	22,091
	Gosund Plug 0329	27,302	DDoS-RSTFIN	18,638
	Netatmo Weather Station	27,237	DoS-SYN	16,199
	Gosund Socket 039A	26,995	mirai-Greet	15,844
	Gosund Plug 147F	26,990	DDoS-SYNolP	11,094
	Gosund Plug 10AC	26,492	SQLInjection	7469
	Gosund Socket 1009	25,864	Bhijacking	7129
	Gosund Plug 0C39	25,184	XSS	6513
	Amazon Plug	22,396	Backdoor-Mal	5584
	Yutron Plug 1	22,104	Recon-PingSweep	3711
	Ring Base Station	22,019	Uploading	2691
	Teckin Plug 2	21,440	Recon-PortScan	1017
	Teckin Plug 1	20,343		
	Yutron Plug 2	20,330		
	SIMCAM 1S	19,898		
	HeimVision Lamp	18,874		
	Smart Board	17,799		
	Atomi Coffee Maker	17,583		
	iRobot Roomba	14,830		
	Luohe Cam Dog	11,482		
	Arlo Q Cam (2)	11,285		
	Nest Mini (2)	11,000		
	D-Link Water Sensor	186		

though slightly higher on CICIoT2023. DI models exhibit marginally higher FPRs, particularly 1D-CNN-DI (0.0069 on UNSW2019), but XGB-DI's low FPR (0.0003-0.0004) indicates robust device fingerprint discrimination. The low Φ across models highlights their precision, with XGB-IDS achieving the lowest values (0.0004-0.0005), reflecting accurate attack label predictions, especially on UNSW2019. RF-IDS and DT-IDS show slightly higher but consistent Φ (0.0006-0.0010). DI models, notably 1D-CNN-DI (0.0068 on UNSW2019), exhibit higher Φ , indicating challenges with overlapping device fingerprints in complex datasets like CICIoT2023 with more than 40 classes. XGB-DI and RF-DI perform better, leveraging feature importance to mitigate mislabeling. These results, further detailed in the confusion matrix in Table 8 for CICIoT2023, underscore iPASecIoT's robustness and its potential to enhance IoT security through efficient and accurate multioutput classification.

4.2. Evaluation of model performance based on individual features

To assess the discriminative power of individual features for iPASecIoT's dual-output functionality (DI and IDS), we evaluate each feature's ability to independently classify devices and network traffic. Results from the XGB classifier on the CICIoMT2024 dataset, presented in Table 9, reveal varying levels of feature effectiveness, measured by Acc and MCC. Highly discriminative features include TCP_w_size (Device: Acc 0.8513, MCC 0.7629; Traffic Type: Acc 0.5394, MCC 0.4901), TCP_ack (Device: Acc 0.8461, MCC 0.7527; Traffic Type: Acc 0.8194, MCC 0.7972), Pck_Size_Q3 (Device: Acc 0.8384, MCC 0.7375; Traffic Type: Acc 0.6054, MCC 0.5454), and Pck_Size_Avg (Device: Acc 0.8228, MCC 0.7181; Traffic Type: Acc 0.7791, MCC 0.7403).

These features TCP_w_size, TCP_ack, Pck_Size_Q3, and Pck_Size_Avg demonstrate robust discriminative capacity, effectively capturing device-specific communication patterns and traffic dynamics. In contrast, HTTP-based features, such as HTTP_method,

Table 7

Distribution of devices and traffic types across UNSW2019, and IoMT2024 datasets and the number of samples for each class.

Dataset	Device Distribution		Traffic Distribution	
	Device	Samples	Traffic Type	Samples
UNSW2019	TPLink Router	1,443,306	Benign	234,657
	Dropcam	675,110	Attack	5,645,463
	Belkin Motion Sensor	225,216		
	Samsung SmartCam	44,437		
	Netatmo Welcome	28,743		
	Wemo Switch	21,649		
	Smart Things	18,091		
	Amazon Echo	11,976		
	TP-Link Smart Plug	11,740		
	Galaxy Tab	6920		
	LiFX Bulb	6109		
	iHome	4111		
	TP-Link Camera	2794		
	HP Printer	2514		
	Triby Speaker	2508		
	PIX-STAR Frame	1728		
	Netatmo Weather Station	1316		
	Ecobee Camera	100,925	Benign	45,856
	Plugable Tech	41,181	ARP Spoofing	35,709
	iPad	9505	MQTT-Malformed Data	33,428
	Tuya Owltron	8128	TCPIP-DoS-ICMP1	25,296
CICIoMT2024	Sense-U Monitor	6704	Recon-PortScan	24,423
	Blink Mini	6092	Recon-OSScan	20,776
	MIT Laxihub	4910	Recon-VulScan	15,091
	Tuya Pager	4443	TCPIP-DDoS-UDP1	357
	Rhythm + HR	2911	TCPIP-DoS-UDP1	341
	Checkme BP2A	2502	Recon-PingSweep	332
	Checkme O2	2323	MQTT-DDoS-Publish	298
	Tuya SOS Button	2021	TCPIP-DDoS-ICMP1	189
	Lookee Ring	1980	MQTT-DoS-Publish	88
	CheckmeO2	1460	MQTT-DoS-Connect	71
	Wellue Ring	1445	MQTT-DDoS-Connect	66
	HR Monitor	1386	TCPIP-DDoS-SYN1	1
	Lookee O2 Ring	1356		
	SleepU Monitor	1307		
	COOSPO Armband	1213		
	TPLink	346		
	Realtek Semic	184		

HTTP_user_agent, HTTP_host, HTTP_content_type (Device: Acc $\approx$ 0.5473, MCC $\approx$ 0.0070; Traffic Type: Acc $\approx$ 0.1937, MCC $\approx$ 0.0042), and DNS-related features, including DNS_Server, DNS_Query_Type, DNS_Query_Len, DNS_Interval, DNS_Answer_Len (Device: Acc $\approx$ 0.549, MCC $\approx$ 0.006-0.053; Traffic Type: Acc $\approx$ 0.194, MCC $\approx$ 0.002-0.009), exhibit lower effectiveness. These features have limited impact on model predictions, likely due to their static nature or infrequent occurrence in IoT traffic.

Highly effective features, such as TCP_w_size, TCP_ack, and Pck_Size_Q3, capture critical device-specific patterns, including TCP window scaling dynamics and packet size distributions, which are unique to IoT device behaviors. Moderately effective features, like IAT_mean and Pck_size, indicate that temporal and volumetric characteristics provide useful but less distinctive information, possibly due to overlap across devices. Low-effectiveness features, such as TCP_flags and HTTP/DNS attributes, suggest that protocol-specific or application-layer features are less relevant for DI and IDS in this dataset, likely due to their limited variability or context-specific applicability. These findings highlight iPASecIoT's ability to leverage a reduced set of highly discriminative features, enhancing efficiency and robustness in IoT security applications.

The dataset's feature space reveals a contrast between packet-level features (e.g., packet sizes, TCP attributes) and application-layer features (e.g., HTTP, DNS). Packet-level features dominate due to their direct relevance to IoT traffic patterns, while application-layer features are less effective, possibly due to sparse data or lack of attack-specific signatures. The high performance of statistical features (e.g., Pck_Size_Q3, Pck_Size_Avg) suggests that aggregated metrics capture robust patterns, whereas binary or static features (e.g., IP_DF, TCP_CWR) fail to provide discriminative power.

4.3. Model performance evaluation using features-based aggregation score (votes)

This section provides a comprehensive analysis of the features selected by the FOA-CKSV described in Algorithm 1 and its impact on the performance of the iPASecIoT model. The evaluation leverages the CICIoT2023 dataset that includes benign activities and

Table 8
Confusion matrix result of XGB prediction on the CiCIoT2023 dataset in classifying the network traffic types.

True/Pred	Backdoor-Mal	Benign	Bhijacking	DDoS-ACKF	DDoS-HTTP	DDoS-ICMP	DDoS-ICMP	DDoS-PSHAC	DDoS-RSTFI	DDoS-SYN	DDoS-SYNol	DDoS-SlowL	DDoS-TCPF	DDoS-UDPF	DDoS-UDPF	DNS-SPF	DoS-HTTP	DoS-SYN	DoS-TCPF	DoS-UDPF	MITM-ARPS	Mitral-Gree	Mitral-Gree	Mitral-Ldpp	Recon-OSSc	Recon-Ping	Recon-Port	SQLInjection	Uploading	Vul-Scan	XSS
Backdoor-Mal	1314	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Benign	0	10478	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bhijacking	0	0	1569	0	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DDoS-ACKF	0	0	0	7084	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DDoS-HTTP	0	0	0	0	10787	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DDoS-ICMP	0	0	0	0	0	6682	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DDoS-ICMP	0	0	0	0	0	0	6286	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DDoS-PSHAC	0	0	0	0	0	0	0	6966	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DDoS-RSTFI	0	0	0	0	0	0	0	0	4564	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DDoS-SYN	0	0	0	0	0	0	0	0	0	4199	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DDoS-SYNol	0	0	0	0	0	0	0	0	0	0	2787	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DDoS-SlowL	0	0	0	0	0	0	0	0	0	0	0	10254	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DDoS-TCPF	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DDoS-UDPF	0	0	0	0	0	0	0	0	0	0	0	0	0	0	5278	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DNS-SPF	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	5866	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DoS-HTTP	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9259	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DoS-SYN	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	3510	0	0	0	0	0	0	0	0	0	0	0	0	0
DoS-TCPF	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	6	0	0	0	0	0	0	0	0	0	0	0	0
DoS-UDPF	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	5026	0	0	0	0	0	0	0	0	0	0	0
MITM-ARPS	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Mitral-Gree	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Mitral-Gree	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Mitral-Ldpp	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Recon-OSSc	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Recon-Ping	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Recon-Port	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
SQLInjection	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Uploading	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Vul-Scan	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
XSS	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

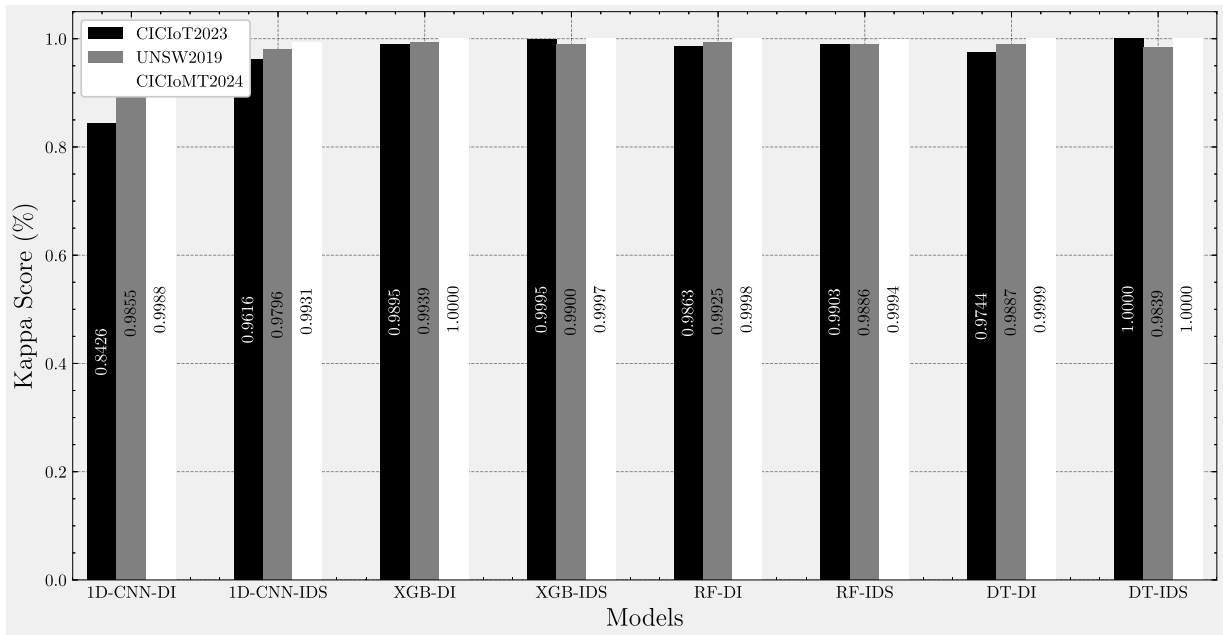


Fig. 8. Comparison of Kappa Scores (%) achieved by various models on the three datasets, illustrating iPASEcIoT's effectiveness in classifying devices and network traffic.

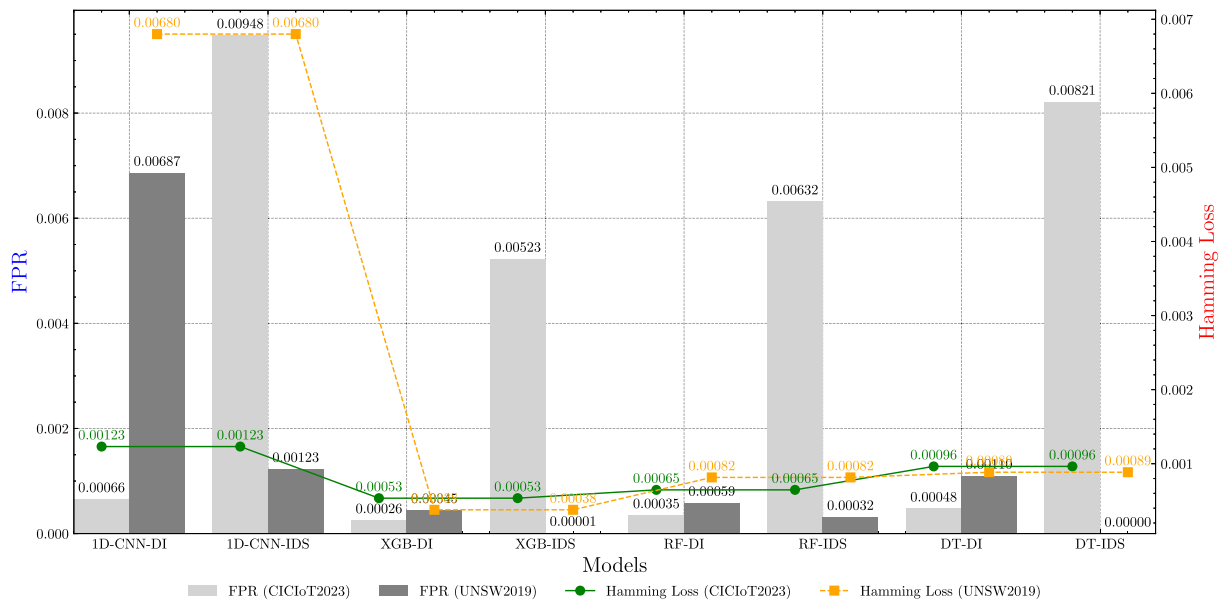


Fig. 9. Performance evaluation of models based on Hamming Loss and False Positive Rate (FPR) for CICIoT2023 and UNSW2019. The line plot illustrates Hamming Loss values for iPASEcIoT.

various intrusion attempts such as SYN flood attacks, DoS ICMP, brute force, and command injection, and several devices, including Amazon Echo dot, Amazon Alexa Spot, Amazon plug, Nest mini, D-Link water sensor etc.

4.3.1. Voting thresholds analysis of FOA-CKSV

The FOA-CKSV categorizes features according to their aggregated votes, with higher votes indicating greater discriminative power. The following thresholds were analyzed. With $Votes = 7$, 22 features covering protocol-related ('BOOTP_options', 'DHCP_options', 'DNS_aa') and statistic features such as ('Burst_count', 'Entropy_Var', 'IAT_mean'). With $Votes \geq 8$, 13 features with robust significance were identified covering protocol, application layer and traffic statistics, and only 'Min_Elapsed_Time' and 'Pck_Size_Max' were

Table 9

Performance metrics for XGBoost multioutput classifier by feature on CICIoMT2024.

Feature	Device				Traffic Type			
	Acc	Kappa	MCC	F1 Score	Acc	Kappa	MCC	F1 Score
TCP_w_size	0.8513	0.7436	0.7629	0.4529	0.5394	0.4553	0.4901	0.2099
TCP_ack	0.8461	0.7365	0.7527	0.5498	0.8194	0.7873	0.7972	0.3976
Pck_Size_Q3	0.8384	0.7286	0.7375	0.5404	0.6054	0.5362	0.5454	0.3445
TCP_dport	0.8375	0.7152	0.7404	0.5188	0.6877	0.6305	0.6520	0.3204
Pck_Size_Avg	0.8228	0.7172	0.7181	0.5231	0.7791	0.7398	0.7403	0.3576
Pck_size	0.8148	0.6891	0.6973	0.4231	0.5898	0.5171	0.5273	0.2721
Entropy	0.6203	0.2645	0.3459	0.3714	0.3457	0.2287	0.2651	0.1667
TCP_flags	0.5744	0.1114	0.1735	0.0945	0.4683	0.3691	0.4073	0.1528
IP_DF	0.5472	0.0000	0.0000	0.0643	0.2595	0.1072	0.1387	0.0542
HTTP_method	0.5473	0.0001	0.0070	0.0645	0.1937	0.0000	0.0042	0.0217

selected at $Votes = 10$. The Acc, and time analysis for each of these scenarios is presented in Table 10 while the features used for each of the scenarios is presented in Table 11.

4.3.2. Impact of features on device identification and intrusion detection

The selected features as in Table 11 influence the performance of iPASecIoT in distinguishing IoT devices and detecting malicious activities. For example, 'Min_Elapsed_Time' ($Votes = 10$) reflects the minimum inter-packet delay, enabling differentiation of devices based on communication timing patterns. It shows its criticality for detecting SYN flood attacks (rapid packet bursts) and DoS ICMP (high-frequency ICMP requests), where short elapsed times indicate malicious intent. Its high vote suggests stability across attack scenarios. Similarly, 'Pck_Size_Max' ($Votes = 10$) captures maximum packet sizes unique to device traffic profiles (e.g., large payloads for video devices vs. small for sensors). The feature also detects brute force attacks (unusually large authentication packets) and command injection (oversized payloads for malicious commands). Its prominence highlights its role in identifying payload anomalies.

The 'UDP_sport' ($Votes \geq 8$) identifies UDP-based DoS attacks (e.g., amplification) by tracking anomalous source ports and 'HTTPS', 'TLS_server_version', 'TLS_version' ($Votes \geq 8$) which are secure protocol versions differentiate devices with encryption support and detects command injection and brute force via TLS version mismatches or weak cipher exploitation in HTTPS traffic. Additionally, the 'TCP_seq' ($Votes \geq 8$) sequence numbers reflect device-specific TCP session patterns and also detects SYN flood attacks by identifying out-of-sequence packets and command injection via manipulated sequences in exploited sessions. Features such as 'Burst_count', 'IAT_mean', 'Time_Since_Prev_Frame' ($votes = 7$), represent the temporal burstiness and inter-arrival times that distinguish device communication rhythms, and effective in detecting SYN floods and DoS ICMP via high burst counts and low inter-arrival times.

4.3.3. Key insights and technical analysis

The temporal and packet-level dominance exhibited by features with vote = 10, such as 'Min_Elapsed_Time' and 'Pck_Size_Max' ($votes = 10$), elaborates their criticality in both DI and ID, making them highly relevant in detecting SYN floods (short delays), DoS ICMP (high packet rates), and brute force (large packets). Their stability across thresholds underscores their robustness. Also, protocol-specific sensitivity fingerprints such as 'Protocol_ICMP_Ratio', 'TCP_dport', and 'TCP_seq' are more critical for ID, particularly against SYN floods and DoS ICMP, due to their sensitivity to protocol anomalies. 'ARP' and 'DNS_rd' enhance detection of spoofing and amplification attacks. Application layer fingerprints like 'HTTP_status' and TLS-related features are effective against command injection and brute force, leveraging status codes and version mismatches as attack signatures while features like 'Entropy_Var' and 'IAT_mean' offer untapped potential for ID, particularly for command injection and SYN floods, if enhanced through feature engineering. However, although the two features with votes = 10 shows dominance, they exhibit limitations when used alone, hence must be integrated with other features. As shown in Table 10, the CNN-based models perform below the ML models. Hence, models trained with all selected features and vote = 8 yield better performance results. In general, XGB stands out among all models with a performance accuracy of 99.02 % for DI and 99.96 % for IDS for all features. Similarly, for votes ≥ 8 (13 features), XGB achieved 96.14 % for DI and 99.85 % for IDS, hence, the XGB model was selected among the evaluated models for further analysis.

4.3.4. Computational implication analysis

The number of features significantly impacts computational cost. As shown in Table 10, 1D-CNN shows exponential train time growth (911.445 s at 22 features, 3418.341 s at 13 features (20 epochs), and 1994.530 s with all features) due to its reliance on feature dimensionality, with per-sample train time varying from 0.002157 s to 0.010241 s, then 0.004713 s, and test time per sample from 0.000047 s to 0.000575 s, then 0.000270 s, indicating optimal efficiency with all features. On the other hand, tree-based models (XGB, RF, DT) exhibit linear or sublinear scaling. XGB train time rises from 57.804 s to 228.866 s, then drops to 216.547 s, with per-sample time from 0.000137 s to 0.000686 s, then 0.000512 s, showing efficient feature use. RF scales poorly (171.872 s to 773.336 s), with per-sample time increasing from 0.000407 s to 0.001827 s due to ensemble complexity, while DT remains efficient (peak 82.256 s, per-sample 0.000194 s). The minimal feature set ($votes = 10$, 2 features) reduces load (e.g., DT 2.217 s) but lowers the precision of the DI, reflecting an efficiency-performance trade-off. 1D-CNN's absence at votes = 10 highlights its need for more features for effective convolution.

Table 10

Performance metrics across feature sets and models using the CICIoT2023 dataset.

FOA-CKSV	Model	Task	Accuracy	T-T (s)	t-t (s)	T-T/Sample (s)	t-t/Sample (s)
All Selected Features	1D-CNN-DI	DI	0.8530	1994.530	49.049	0.004713	0.000270
	1D-CNN-IDS	IDS	0.9634	1994.530	49.049	0.004713	0.000270
	XGB-DI	DI	0.9902	216.547	8.982	0.000512	0.000049
	XGB-IDS	IDS	0.9996	216.547	8.982	0.000512	0.000049
	RF-DI	DI	0.9873	773.336	55.146	0.001827	0.000304
	RF-IDS	IDS	0.9907	773.336	55.146	0.001827	0.000304
	DT-DI	DI	0.9761	82.256	0.694	0.000194	0.000004
	DT-IDS	IDS	1.0000	82.256	0.694	0.000194	0.000004
Vote = 7 (22 features)	1D-CNN-DI	DI	0.7321	911.445	8.576	0.002157	0.000047
	1D-CNN-IDS	IDS	0.9565	911.445	8.576	0.002157	0.000047
	XGB-DI	DI	0.8972	57.804	3.690	0.000137	0.000020
	XGB-IDS	IDS	0.8407	57.804	3.690	0.000137	0.000020
	RF-DI	DI	0.9229	171.872	27.261	0.000407	0.000151
	RF-IDS	IDS	0.8977	171.872	27.261	0.000407	0.000151
	DT-DI	DI	0.9059	47.858	0.718	0.000113	0.000004
	DT-IDS	IDS	0.8645	47.858	0.718	0.000113	0.000004
Vote ≥ 8 (13 features)	1D-CNN-DI	DI	0.7147	3418.341	82.263	0.010241	0.000575
	1D-CNN-IDS	IDS	0.9659	3418.341	82.263	0.010241	0.000575
	XGB-DI	DI	0.9614	228.866	10.851	0.000686	0.000076
	XGB-IDS	IDS	0.9985	228.866	10.851	0.000686	0.000076
	RF-DI	DI	0.9608	280.278	39.704	0.000840	0.000278
	RF-IDS	IDS	0.9989	280.278	39.704	0.000840	0.000278
	DT-DI	DI	0.9588	13.432	0.415	0.000040	0.000003
	DT-IDS	IDS	1.0000	13.432	0.415	0.000040	0.000003
Vote = 10 (2 features)	1D-CNN-DI	DI	0.5520	1402.85	20.69	0.008735	0.000301
	1D-CNN-IDS	IDS	0.9338	1402.85	20.69	0.008735	0.000301
	XGB-DI	DI	0.7971	63.586	4.198	0.000396	0.000061
	XGB-IDS	IDS	0.9873	63.586	4.198	0.000396	0.000061
	RF-DI	DI	0.8497	92.601	5.985	0.000577	0.000087
	RF-IDS	IDS	0.9997	92.601	5.985	0.000577	0.000087
	DT-DI	DI	0.8520	2.217	0.313	0.000014	0.000005
	DT-IDS	IDS	1.0000	2.217	0.313	0.000014	0.000005

Table 11

Description of features obtained by voting analysis on the CICIoT2023 dataset.

FS Criteria	Selected Features
Vote = 7	'BOOTP_options', 'Burst_count', 'DHCP_options', 'DNS_aa', 'Direction_ratio', 'Entropy_Q1', 'Entropy_Var', 'IAT_mean', 'ICMP_chksum', 'IP_MF', 'IP_chksum', 'MQTT_type', 'Pck_Size_Q1', 'Pck_Size_Q3', 'Pck_Size_Var', 'Pck_size', 'Portcl_dst', 'TCP_FIN', 'TCP_URG', 'TCP_sport', 'Time_Since_Prev_Frame', 'sport23'
Vote ≥ 8	'ARP', 'DNS_rd', 'HTTPS', 'HTTP_status', 'Min_Elapsed_Time', 'Payload_Len_Q1', 'Pck_Size_Max', 'Protocol_ICMP_Ratio', 'TCP_dport', 'TCP_seq', 'TLS_server_version', 'TLS_version', 'UDP_sport'
Vote = 10	'Min_Elapsed_Time', 'Pck_Size_Max'

The results presented highlights the lightweight nature of the proposed methods. The DT model, with a peak training time of 82.256 s and a per-sample inference time of 0.000194 s, exemplifies rapid processing, making it highly suitable for edge devices. Similarly, the XGB model, with training times ranging from 57.804 s to 228.866 s and per-sample inference times from 0.000137 s to 0.000686 s, offers a balance of speed and scalability, further contributing to its lightweight profile. In contrast, RF model, despite its higher training time (up to 773.336 s) and per-sample inference time (0.001827 s), remains viable for applications prioritizing robustness over speed. The IDCNN model, with training times up to 3418.341 s and inference times ranging from 0.000047 s to 0.000575 s, provides fast inference despite its higher training demands, enhancing its suitability for real-time applications. Additionally, memory usage measurements in terms of model size on disk further illustrates the lightweight characteristics of the proposed method. Specifically, DT, XGB, RT and 1D-CNN use 11 MB, 10 MB, 1.8 GB due to its ensemble complexity, and 11.2 MB, respectively. These metrics collectively demonstrate that DT and XGB, with their low training and inference times coupled with minimal memory footprints, are particularly lightweight and well-suited for efficient deployment in resource-limited settings. It is noteworthy that the memory usage is influenced by the model architecture and feature space. Hence, by optimizing the model architecture and feature space, we further reduced the complexity of our model.

Table 12 presents the precision, recall, and F1 score per class for the iPASeIoT model, evaluated on the CICIoT2023 and CICIoT2024 datasets for attack classification. These results, obtained using the classification report function in scikit-learn, provide a detailed view of the model's ability to accurately identify each attack category alongside benign traffic. For example, as shown in

Table 12, in classifying backdoor malware samples, the machine learning models achieved precision, recall, and F1 scores of 100 %, while the CNN-based model achieved 74 %, 11 %, and 19 % for precision, recall, and F1 score, respectively, on the CICIOT2023 dataset. Conversely, the models detect DoS TCP Flood attack samples with 100 % precision, recall, and F1 scores. This analysis provides granular insights into the model's performance across different attack categories, beyond the aggregate accuracy reported in **Table 10**. These metrics help identify strengths and weaknesses, such as whether certain attacks, like backdoor malware or DoS attempts, are more easily detected than stealthier attacks, such as command injection. Since the task involves multiclass classification, these per-class metrics support the claim that iPASecIoT effectively distinguishes a wide range of IoT-specific threats in the CICIOT2023 and CICIOT2024 datasets, reinforcing its applicability in real-world intrusion detection scenarios.

Similarly, **Table 13** presents the per-class evaluation of iPASecIoT's ability to identify various devices in the datasets, highlighting its dual capability and robustness. Specifically, seven devices from each dataset were analyzed. The XGB model identifies the Amazon Plug device in CICIOT2023 with precision, recall, and F1 scores of 99.44 %, 99.47 %, and 99.45 %, respectively. The Ecobee Camera is recognized with 99.98 %, 100 %, and 99.99 % for precision, recall, and F1 score, respectively, when the XGB model is evaluated using samples from the CICIOT2024 dataset. Additionally, when evaluating the model's predictive ability for identifying device types in the UNSW2019 dataset, XGB achieves 98.68 %, 97.77 %, and 98.22 % for precision, recall, and F1 score, respectively, on Belkin Motion Sensor samples.

4.4. Analysis of iPASecIoT performance on tuned device-specific and traffic-specific fingerprints

In this subsection, we evaluate iPASecIoT's ability to adaptively leverage device-specific and traffic-specific fingerprints for accurate DI and IDS, validating the efficacy of the FOA-CKSV FS process. The evaluation uses the CICIOT2024 dataset, with results presented in **Tables 14** and **15** for DI and IDS, respectively.

Device-specific feature usage enhances iPASecIoT's performance by tailoring features to the unique network behaviors of each IoMT device. For example, the Blink Mini model prioritizes Protocol_UDP_Ratio and Protocol_TCP_Ratio, reflecting its reliance on protocol-specific patterns. Similarly, the Ecobee Camera model emphasizes IAT_mean and IAT_std, highlighting the importance of inter-arrival time characteristics for accurate identification. Conversely, traffic-specific FS optimizes IDS by distinguishing benign traffic from various attack types. For example, to classify Benign, the model heavily weights Min_Elapsed_Time (importance: 0.241), indicating that timing characteristics are critical for identifying normal traffic. In contrast, attack samples like MQTT-DDoS-Connect_Flood and MQTT-DoS-Connect_Flood are detected by prioritizing Pck_Size_Sum and Pck_Size_Min, reflecting packet size anomalies typical of these attacks.

By utilizing a reduced set of highly relevant features, iPASecIoT minimizes the processing of redundant or less informative features, significantly reducing computational complexity. For device-specific models, training time decreased to 354.17 seconds (**Table 15**) from 773.336 seconds using all features (**Table 10**), demonstrating efficient resource utilization. Similarly, inference time for the test dataset reduced from 55.146 seconds (all features) to 3.8637 seconds (device-specific). The frequent recurrence of Min_Elapsed_Time across both DI and IDS models, with high importance scores (e.g., 0.241 for Benign, 0.070 for MIT Camera Laxihub), aligns with findings in **Section 4.3.3**. This underscores the critical role of timing-based features in IoMT security, likely due to the real-time communication requirements of medical devices. These results highlight iPASecIoT's ability to dynamically adapt features, enhancing efficiency and robustness in IoMT security applications.

4.5. Comparative analysis

iPASecIoT is compared with related works to highlight the scalability, robustness, and efficiency of our proposed method. While no existing studies apply the same fingerprints for simultaneous DI and IDS tasks through a single, we compare iPASecIoT with independent DI and IDS methods from the literature, as shown in **Table 16**. For DI, iPASecIoT exhibits comparable performance to Rabbani et al. [43], which achieved 99.95 % accuracy on CICIOT2022, despite iPASecIoT handling a larger number of devices and traffic types across multiple datasets. Compared to IoTDevID [17], which utilized a genetic algorithm for feature selection and achieved 94.00 % accuracy on the Aalto dataset, iPASecIoT demonstrates superior performance. For IDS, Rabbani et al. [35] achieved 99.94 % accuracy on CICIOT2023 using a lightweight model with behavioral features, closely matching iPASecIoT's 99.9 % accuracy. However, their approach is limited to single-output IDS tasks and lacks the multioutput capability of iPASecIoT. Similarly, Neto et al. [44] reported 98.61 % accuracy on CICIOT2023 with a focus on flow-based features, but their method struggles with scalability in diverse network environments. Zohourian et al. [55] achieved 98.74 % accuracy on CICIOT2023, leveraging statistical and protocol-based features, yet it requires continuous retraining to adapt to evolving threats, increasing computational overhead. The adaptability and generalizability of iPASecIoT to both DI and IDS using a unified set of fingerprints, coupled with high accuracy across CICIOT2023, CICIOT2024, and UNSW2019, underscores its uniqueness compared to the reviewed literature.

5. Conclusion

IoT DI and IDS are critical aspects of IoT security necessary for ensuring secure communication in IoT ecosystems. DI models help to identify compromised devices on a network through their behavior instead of MAC or IP addresses and isolate the such that a successful attack does not quickly spread through the network. On the other hand, IDS monitors the traffic from these devices for possible anomaly. The integration of these two techniques ensure that a higher level of security is maintained. While previous studies have addressed these tasks independently, employing distinct models that are often resource-intensive models and requires using

Table 12
Performance analysis of the various models in detecting the different samples of threats in the datasets. The analysis focuses on the precision, recall and F1.

CICIoT2023						CICIoT2023						CICIoMT2024					
Name	ML	Pre	Rec	F1		Name	ML	Pre	Rec	F1		Name	ML	Pre	Rec	F1	
Backdoor-Mal	DT	1.000	1.000	1.000		Benign	DT	1.000	1.000	1.000		ARP Spoofing	DT	1.000	1.000	1.000	
	RT	1.000	1.000	1.000			RT	0.990	0.990	0.990			RT	1.000	1.000	1.000	
	XGB	1.000	1.000	1.000			XGB	1.000	1.000	1.000			XGB	1.000	1.000	1.000	
DDoS-ACKF	1D-CNN	0.740	0.110	0.190		DDoS-HITP	1D-CNN	1.000	1.000	1.000		Benign	1D-CNN	1.000	0.999	1.000	
	DT	1.000	1.000	1.000			DT	1.000	1.000	1.000			DT	1.000	1.000	1.000	
	RT	0.990	0.990	0.990			RT	0.990	0.990	0.990			RT	1.000	1.000	1.000	
DDoS-ICMP-Fr	XGB	1.000	1.000	1.000		DDoS-PSHACK	XGB	1.000	1.000	1.000		MQTT-DDoS-CF	XGB	1.000	1.000	1.000	
	1D-CNN	0.980	0.980	0.980			1D-CNN	1.000	1.000	1.000			1D-CNN	0.988	1.000	0.999	
	DT	1.000	1.000	1.000			DT	1.000	1.000	1.000			DT	1.000	1.000	1.000	
DDoS-SYN	RT	0.990	0.990	0.990		DDoS-SYNoIP	RT	0.990	0.990	0.990		MQTT-DDoS-Pub	RT	1.000	1.000	1.000	
	XGB	1.000	1.000	1.000			XGB	1.000	1.000	1.000			XGB	0.949	1.000	0.978	
	1D-CNN	1.000	1.000	1.000			1D-CNN	0.980	0.550	0.700			1D-CNN	0.838	1.000	0.779	
DDoS-TCPF	DT	1.000	1.000	1.000		DDoS-UDPF	DT	1.000	1.000	1.000		Recon-OSScan	DT	1.000	1.000	1.000	
	RT	0.990	0.990	0.990			RT	0.990	0.980	0.990			RT	1.000	1.000	1.000	
	XGB	1.000	1.000	1.000			XGB	1.000	1.000	1.000			XGB	1.000	1.000	1.000	
DNS-SPF	1D-CNN	1.000	0.990	0.990		DoS-HTTP	1D-CNN	1.000	0.990	1.000		Recon-VulScan	1D-CNN	0.999	0.999	0.999	
	DT	1.000	1.000	1.000			DT	1.000	1.000	1.000			DT	1.000	1.000	1.000	
	RT	0.990	1.000	0.990			RT	0.990	0.990	0.990			RT	1.000	1.000	1.000	
DoS-TCPF	XGB	1.000	1.000	1.000		DoS-UDPF	XGB	1.000	1.000	1.000		TCP-DDoS-ICMP	XGB	1.000	1.000	1.000	
	1D-CNN	0.870	1.000	0.930			1D-CNN	0.820	0.980	0.890			1D-CNN	0.999	0.999	0.999	
	DT	1.000	1.000	1.000			DT	1.000	1.000	1.000			DT	1.000	1.000	1.000	
DoS-TCPF	RT	1.000	1.000	1.000		DoS-UDPF	RT	0.990	0.990	0.990		TCP-DDoS-ICMP	RT	1.000	1.000	1.000	
	XGB	1.000	1.000	1.000			XGB	1.000	1.000	1.000			XGB	1.000	1.000	1.000	
	1D-CNN	1.000	1.000	1.000			1D-CNN	0.980	0.810	0.880			1D-CNN	1.000	1.000	1.000	

Table 13
Performance analysis of the models based on precision, recall and F1 score on DI.

CICIoT2023					CICIoMT2024					UNSW2019				
Name	ML	Pre	Rec	F1	Name	ML	Pre	Rec	F1	Name	ML	Pre	Rec	F1
Amcrest WiFi Camera	DT	0.9870	0.9839	0.9855	Blink Mini	DT	1.0000	0.9978	0.9989	Amazon Echo	DT	0.8768	0.8958	0.8862
	RT	0.9899	0.9947	0.9923		RT	1.0000	0.9989	0.9995		RT	0.9016	0.9016	0.9016
	XGB	0.9944	0.9947	0.9945		XGB	1.0000	0.9984	0.9992		XGB	0.9447	0.8900	0.9165
Amazon Alexa Echo Dot	1D-CNN	0.8225	0.8305	0.8264	Armband	1D-CNN	0.7840	0.6841	0.7671	Belkin Wemo Switch	1D-CNN	0.9492	0.5104	0.8743
	DT	0.9543	0.9611	0.9577		DT	1.0000	1.0000	1.0000		DT	0.9708	0.9691	0.9699
	RT	0.9766	0.9678	0.9722		RT	0.9891	0.9973	0.9932		RT	0.9834	0.9613	0.9723
Amazon Plug	XGB	0.9792	0.9732	0.9762	Oxygen Oximeter	XGB	1.0000	1.0000	1.0000	Belkin Motion Sensor	XGB	0.9949	0.9658	0.9801
	1D-CNN	0.7814	0.8193	0.7999		1D-CNN	0.7684	0.7582	0.7240		1D-CNN	0.9738	0.9277	0.9502
	DT	0.9536	0.9362	0.9449		DT	1.0000	1.0000	1.0000		DT	0.9683	0.9707	0.9695
Arlo Q Camera	RT	0.9550	0.9654	0.9601	Ecobee Camera	RT	0.7844	0.8721	0.8259	Dropcam	RT	0.9728	0.9721	0.9749
	XGB	0.9609	0.9627	0.9618		XGB	0.9977	1.000	0.9989		XGB	0.9868	0.9777	0.9822
	1D-CNN	0.9008	0.8852	0.8930		1D-CNN	0.8890	0.8690	0.7950		1D-CNN	0.9494	0.9490	0.9492
Arlo Base Station	DT	0.9907	0.9963	0.9935	Baby Monitor	DT	0.9998	0.9996	0.9997	Netatamo Welcome	DT	0.9985	0.9987	0.9986
	RT	0.9842	0.9894	0.9868		RT	0.9998	0.9999	0.9999		RT	0.9991	0.9989	0.9988
	XGB	0.9931	0.9876	0.9903		XGB	0.9998	1.000	0.9999		XGB	0.9994	0.9986	0.9990
Google Nest Mini	1D-CNN	0.9508	0.9515	0.9513	Sleep Ring	1D-CNN	0.7960	0.7550	0.7060	Smart Things	1D-CNN	0.9995	0.9984	0.9990
	DT	0.9559	0.9396	0.9477		DT	0.9936	0.9985	0.9960		DT	0.9524	0.9550	0.9537
	RT	0.9679	0.9645	0.9662		RT	0.9990	1.0000	0.9995		RT	0.9638	0.9600	0.9619
Netatamo Camera	XGB	0.9752	0.9773	0.9762	HR Monitor	XGB	0.9980	1.000	0.9990	Smart Plug	XGB	0.9796	0.9611	0.9703
	1D-CNN	0.9013	0.8558	0.8780		1D-CNN	0.7540	0.7430	0.7420		1D-CNN	0.9547	0.9245	0.9394
	DT	0.9736	0.9713	0.9724		DT	1.0000	1.0000	1.0000		DT	0.9073	0.9278	0.9174
	RT	0.9831	0.9887	0.9859		RT	0.8874	0.9158	0.9014		RT	0.9651	0.9368	0.9507
	XGB	0.9951	0.9924	0.9937		XGB	1.0000	0.9989	0.9990		XGB	0.9623	0.9345	0.9485
	1D-CNN	0.8459	0.8472	0.8466		1D-CNN	0.8120	0.7980	0.7860		1D-CNN	0.9786	0.9278	0.9525
	DT	0.9732	0.9696	0.9714		DT	1.0000	1.0000	1.0000		DT	0.6000	0.6667	0.6316
	RT	0.9925	0.9844	0.9885		RT	0.9852	0.9615	0.9732		RT	0.7209	0.6889	0.7045
	XGB	0.9933	0.9916	0.9924		XGB	1.0000	1.0000	1.0000		XGB	0.7949	0.6889	0.7381
	1D-CNN	0.9296	8769	9025		1D-CNN	0.7940	0.7810	0.7850		1D-CNN	1.0000	0.4444	0.6154

Table 14
Device-specific top features and model performance on the CICIoMT2024 dataset.

Device	Top 5 Features	Importance	Accuracy
Blink mini	Protocol_UDP_Ratio, Protocol_TCP_Ratio, IP_add_count, TTL_default, Min_Elapsed_Time	0.079, 0.078, 0.075, 0.067, 0.060	1.000
Ecobee Camera	IAT_mean, IAT_std, Packet_freq, TTL_default, IP_ttl	0.093, 0.078, 0.064, 0.053, 0.039	0.99997
Sense-U Baby Monitor	TTL_default, Protocol_UDP_Ratio, IP_ttl, IP_add_count, Pck_Size_Max	0.127, 0.092, 0.082, 0.070, 0.063	0.99984
Ipad	sport_bare, IAT_mean, Portcl_src, Pck_Size_Var, IP_add_count	0.101, 0.094, 0.083, 0.072, 0.061	1.000
MIT Camera Laxi-hub	dport_bare, Min_Elapsed_Time, Pck_Size_Max, UDP_sport, sport_bare	0.084, 0.070, 0.062, 0.053, 0.048	1.000

Table 15
Traffic-specific top features and model performance on the CICIoMT2024 dataset.

Traffic Type	Top 5 Features	Importance	Accuracy
Benign	Min_Elapsed_Time, IAT_mean, Pck_Size_IQR, IAT_std, Pck_Size_Kurt	0.241, 0.097, 0.068, 0.062, 0.047	1.000
MQTT-DDoS-Connect_Flood	Pck_Size_Sum, IP_add_count, Min_Elapsed_Time, Pck_Size_Min, IP_Dst_Entropy	0.125, 0.104, 0.098, 0.072, 0.065	0.99997
MQTT-DoS-Connect_Flood	Pck_Size_Sum, Min_Elapsed_Time, Pck_Size_Min, IP_add_count, Pck_Size_Kurt	0.158, 0.115, 0.082, 0.081, 0.066	0.99995
Recon-OS_Scan	Min_Elapsed_Time, Pck_Size_Avg, Payload_Len_Q1, TCP_flags, Payload_Len_Q3	0.115, 0.083, 0.061, 0.052, 0.045	0.99997
TCP_IP-DDoS-ICMP1	Entropy_Var, Pck_Size_Var, Pck_Size_Sum, Pck_Size_Avg, Pck_Size_Max	0.130, 0.077, 0.061, 0.060, 0.052	1.000

Table 16
Comparative analysis of iPASecIoT performance in relation to existing literature.

Reference	Dataset	Output Mode	DI or IDS	Accuracy (IDS)	Accuracy (DI)
IoTDevID [17]	Aalto	Single	DI	–	94.00 %
LFloTDI [22]	CICIoT2022	Single	DI	–	99.08 %
LFloTDI [22]	UNSW2019	Single	DI	–	98.15 %
Rabbani et al. [35]	CICIoT2023	Single	IDS	99.94%	–
Neto et al. [44]	CICIoT2023	Single	IDS	98.61%	–
Zohourian et al. [55]	CICIoT2023	Single	IDS	98.74%	–
TinyDevID [36]	Private	Single	DI	–	91.00 %
Dora et al. [39]	Aalto	Single	DI	–	96.00 %
Rabbani et al. [43]	CICIoT2022	Single	DI	–	99.95 %
	CICIoT2023			99.93 %	99.96 %
	CICIoMT2024			99.97 %	99.94 %
iPASecIoT	UNSW2019	Multioutput	DI & IDS	98.99 %	99.20 %

different feature sets to develop each model, further increasing computational cost, maintenance and processing time. This study introduces iPASecIoT, a single model with multioutput functionality that concurrently performs DI and IDS, leveraging adaptive FS to dynamically utilize the most relevant device- and traffic-specific fingerprints for each sample. iPASecIoT was evaluated on three datasets including, (CICIoT2023, CICIoMT2024, and UNSW2019) encompassing 81 IoT devices and 47 diverse network threats, iPASecIoT achieved an average accuracy of 99.9% and a Hamming loss of ≈ 0.001 , demonstrating its robustness and scalability for enhancing IoT security.

By introducing the enhanced FOA-CKSV, iPASecIoT optimizes the selection of high-impact fingerprints, significantly reducing training and inference times. Validation across diverse scenarios revealed that iPASecIoT can identify devices and classify network traffic with as few as two features. These results highlight iPASecIoT's potency in delivering high-accuracy, resource-efficient solutions for IoT security, marking a significant advancement over prior single-task models. The study evaluated four different algorithms: DT, RF, XGB and 1D-CNN using all selected features and conducted an ablation analysis of different voting options to compare the selected features in a single cross-validation fold with features selected across different folds. In all, XGB achieved superior performance over other algorithms.

However, performance degradation was observed with increasing dataset size, indicating a need for further optimization in large-scale settings. Also, 1D-CNN which was trained for only 10 epochs can show improved performance when evaluated over a longer training time. In the future, we will focus on enhancing iPASecIoT's scalability by extending the training duration of its 1D-CNN to improve accuracy and exploring additional feature scenarios, and include further analysis on the specific contributions of each feature in iPASecIoT's predictions. Furthermore, to address potential data exposure during fingerprint extraction, federated learning will be integrated to enhance data privacy, ensuring compliance with stringent security requirements in real-world IoT deployments.

CRedit authorship contribution statement

Ogobuchi Daniel Okey: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Formal analysis, Data curation, Conceptualization; **Sajjad Dadkhah:** Writing – review & editing, Validation, Supervision, Project administration, Methodology; **Heather Molyneaux:** Writing – review & editing, Visualization, Validation; **Demóstenes Zegarra Rodríguez:** Writing – review & editing, Validation, Supervision; **João Henrique Kleinschmidt:** Writing – review & editing, Validation, Supervision, Project administration.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work is supported by [Fundação de Amparo à Pesquisa do Estado de São Paulo \(FAPESP\)](#) under Grant [2023/07184-1](#) & [2024/12903-0](#)

References

- [1] Statista, Internet of things - worldwide, 2024. [Accessed, April 25, 2025], <https://tinyurl.com/statista2024report>.
- [2] Lueth, Knud Lasse, Insights-Release-State-of-IoT-2023-Number-of-connected-IoT-devices-growing-16-to-16.0-billion-globally, Technical Report, 2023. <https://www.iot-analytics.com>.
- [3] D. Gu, J. Zhang, Z. Tang, Q. Li, M. Zhu, H. Yan, H. Li, IoT device identification based on network traffic, *Wireless Netw.* 31 (2) (2025) 1645–1661.
- [4] S. Dadkhah, E.C.P. Neto, R. Ferreira, R.C. Molokwu, S. Sadeghi, A.A. Ghorbani, CICIoMT2024: a benchmark dataset for multi-protocol security assessment in IoMT, *Internet of Things* 28 (2024) 101351.
- [5] P. He, Y. Zhou, X. Qin, A survey on energy-aware security mechanisms for the internet of things, *Future Internet* 16 (4) (2024). <https://www.mdpi.com/1999-5903/16/4/128>. <https://doi.org/10.3390/fi16040128>
- [6] J. Telo, Smart city security threats and countermeasures in the context of emerging technologies, *Int. J. Intell. Autom. Comput.* 6 (1) (2023) 31–45.
- [7] H. Tahaei, F. Afifi, A. Asemi, F. Zaki, N.B. Anuar, The rise of traffic classification in IoT networks: a survey, *J. Net. Comput. Appl.* 154 (2020) 102538.
- [8] T.H. News, New flaws in TPM 2.0 library pose threat to billions of IoT and enterprise devices — thehackernews.com, 2024, (<https://tinyurl.com/thehackersnews>). [Accessed 28-04-2025].
- [9] T.I. NETSCOUT, Dawn of the terrorbit era | NETSCOUT — netscout.com, 2018, (<https://www.netscout.com/blog/dawn-terrorbit-era>). [Accessed 28-04-2025].
- [10] R. Nazir, A.A. Laghari, K. Kumar, S. David, M. Ali, Survey on wireless network security, *Arch. Comput. Methods Eng.* 29 (3) (2022) 1591–1610.
- [11] I. Hafeez, M. Antikainen, A.Y. Ding, S. Tarkoma, IoT-KEEPER: detecting malicious IoT network activity using online traffic analysis at the edge, *IEEE Trans. Netw. Serv. Manage.* 17 (1) (2020) 45–59.
- [12] A. Marzano, D. Alexander, O. Fonseca, E. Fazzion, C. Hoepers, K. Steding-Jessen, M.H. Chaves, Í. Cunha, D. Guedes, W. Meira, The evolution of bashlite and mirai IoT botnets, in: 2018 IEEE Symposium on Computers and Communications (ISCC), IEEE, 2018, pp. 00813–00818.
- [13] J. Mohanty, S. Mishra, S. Patra, B. Pati, C.R. Panigrahi, IoT security, challenges, and solutions: a review, *Progress Adv. Comput. Intell. Eng. Proc. ICACIE* 2019, Volume 2 (2020) 493–504.
- [14] S. Zhu, X. Lu, Intelligent device identification based on traffic time-series features, in: 3rd International Conference on Internet of Things and Smart City (IoTSC 2023), 12708, SPIE, 2023, pp. 546–550.
- [15] Q. Hao, Z. Rong, IoTTFID: an incremental IoT device identification model based on traffic fingerprint, *IEEE Access* 11 (2023) 58679–58691.
- [16] H. Wang, D. Eklund, A. Oprea, S. Raza, FL4IoT: IoT device fingerprinting and identification using federated learning, *ACM Trans. Internet Things* 4 (3) (2023) 1–24.
- [17] K. Kostas, M. Just, M.A. Lones, IoTDevID: a behavior-based device identification method for the IoT, *IEEE Internet Things J.* 9 (23) (2022) 23741–23749.
- [18] M. Miettinen, S. Marchal, I. Hafeez, N. Asokan, A.-R. Sadeghi, S. Tarkoma, IoT SENTINEL: automated device-type identification for security enforcement in iot, in: 2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS), IEEE, 2017, pp. 2177–2184.
- [19] B. Bezawada, M. Bachani, J. Peterson, H. Shirazi, I. Ray, I. Ray, Behavioral fingerprinting of IoT devices, in: Proceedings of the 2018 Workshop on Attacks and Solutions in Hardware Security, 2018, pp. 41–50.
- [20] A. Bremler-Barr, H. Levy, Z. Yakhini, IoT or not: identifying IoT devices in a short time scale, in: NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium, IEEE, 2020, pp. 1–9.
- [21] A. Sivanathan, H.H. Gharakheili, F. Loi, A. Radford, C. Wijenayake, A. Vishwanath, V. Sivaraman, Classifying IoT devices in smart environments using network traffic characteristics, *IEEE Trans. Mob. Comput.* 18 (8) (2018) 1745–1759.
- [22] Z. Xu, Q. Lu, F. Chen, H. Xian, LFIoTDL: a lightweight and fine-grained device identification approach for IoT security enhancement, *Comput. Commun.* 237 (2025) 108149.
- [23] D. Legkodymov, D. Levshun, Exploring efficiency of machine learning in profiling of internet of things devices for malicious activity detection, in: ICISP 2025 - 11th International Conference on Information Systems Security and Privacy, - Volume 2, 2025, pp. 276–283. <https://doi.org/10.5220/0013389100003899>
- [24] O. Höfener, Q. Wang, Requirements framework for IoT device authentication using behavioral fingerprinting, *Comput. Secur.* 154 (2025) 104459.
- [25] D. Zhang, X. Zhang, H. Wang, Y. Yu, DevPF: device identification through passive fingerprints in IoT, in: 2024 7th International Symposium on Autonomous Systems (ISAS), IEEE, 2024, pp. 1–6.
- [26] S.S. Wanode, M. Anand, B. Mitra, Optimal feature set selection for IoT device fingerprinting on edge infrastructure using machine intelligence, in: IEEE INFOCOM 2022-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), IEEE, 2022, pp. 1–6.
- [27] S. Kumar, M. Das, S. Nandi, D. Goswami, DI4IoT: a comprehensive framework for IoT device-type identification through network flow analysis, *Internet Things* 31 (2025) 101599.
- [28] O.D. Okey, S.S. Maidin, P. Adasme, R. Lopes Rosa, M. Saadi, D. Carrillo Melgarejo, D. Zegarra Rodríguez, BoostedenML: efficient technique for detecting cyberattacks in IoT systems using boosted ensemble machine learning, *Sensors* 22 (19) (2022) 7409.
- [29] G.A. Morales, J. Xu, D. Zhu, R. Slavin, Lightweight collaborative inferencing for real-time intrusion detection in IoT networks, 2022. <https://doi.org/10.1109/SmartWorld-UIC-ATC-ScalCom-DigitalTwin-PriComp-Metaverse56740.2022.00076>

- [30] X.-H. Nguyen, X.-D. Nguyen, H.-H. Huynh, K.-H. Le, RealGuard: a lightweight network intrusion detection system for IoT gateways, *Sensors* 22 (2) (2022) 432.
- [31] O.D. Okey, D.C. Melgarejo, M. Saadi, R.L. Rosa, J.H. Kleinschmidt, D.Z. Rodríguez, Transfer learning approach to IDS on cloud IoT devices using optimized CNN, *IEEE Access* 11 (2023) 1023–1038.
- [32] P. Mani, D. Arthi, K. Periyakaruppan, S. Surenderkumar, A lightweight and federated machine learning based intrusion detection system for multi attack detection in IoT networks, 2025. <https://doi.org/10.53759/7669/jmc202505033>
- [33] Z. Ma, L. Liu, W. Meng, X. Luo, L. Wang, W. Li, ADCL: toward an adaptive network intrusion detection system using collaborative learning in IoT networks, 2023. <https://doi.org/10.1109/JIOT.2023.3248259>
- [34] F. Yin, L. Yang, J. Ma, Y. Zhou, Y. Wang, J. Dai, Identifying IoT devices based on spatial and temporal features from network traffic, *Secur. Commun. Netw.* 2021 (1) (2021) 2713211.
- [35] M. Rabbani, J. Gui, Z. Zhou, F. Nejati, M. Mirani, G. Piya, I. Opushnyev, R. Lu, A. Ghorbani, A lightweight IoT device identification using enhanced behavioral-based features, *Peer-to-Peer Netw. Appl.* 18 (2) (2025) 1–22.
- [36] P.R. Chaudhary, A. Agrawal, R.R. Maiti, TinyDevID: tinyML-driven IoT devices Identification using network flow data, in: 2025 17th International Conference on Communication Systems and Networks (COMSNETS), IEEE, 2025, pp. 1335–1340.
- [37] S. Ding, D.-J. Han, C.G. Brinton, K. Dasala, FICDF: a federated incremental learning framework for IoT device fingerprinting, in: 2024 22nd International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt), IEEE, 2024, pp. 337–344.
- [38] O. Pospisil, R. Fudjak, Identification of industrial devices based on payload, in: Proceedings of the 19th International Conference on Availability, Reliability and Security, 2024, pp. 1–9.
- [39] S. Dora, B. Peshvani, A. Kumar, P. Tiwari, S. Venkatesan, S.K. Shukla, R. Khondoker, Automated IoT device identification based on full packet information using real-time network traffic, in: 2025 17th International Conference on Communication Systems and Networks (COMSNETS), IEEE, 2025, pp. 144–149.
- [40] M.V. Shenoy, et al., HFedDI: a novel privacy preserving horizontal federated learning based scheme for IoT device identification, *J. Netw. Comput. Appl.* 214 (2023) 103616.
- [41] J. Zhao, Q. Li, J. Sun, M. Dong, K. Ota, M. Shen, Efficient IoT device identification via network behavior analysis based on time series dictionary, *IEEE Internet Things J.* 11 (3) (2023) 5129–5142.
- [42] Q. Chen, Y. Song, B. Jennings, F. Zhang, B. Xiao, S. Gao, IoT-ID: Robust IoT device identification based on feature drift adaptation, in: 2021 IEEE Global Communications Conference (GLOBECOM), IEEE, 2021, pp. 1–6.
- [43] M. Rabbani, J. Gui, F. Nejati, Z. Zhou, A. Kaniyamattam, M. Mirani, G. Piya, I. Opushnyev, R. Lu, A.A. Ghorbani, Device identification and anomaly detection in IoT environments, *IEEE Internet Things J.* 12 (10) (2025) 13625–13643. <https://doi.org/10.1109/JIOT.2024.3522863>
- [44] E.C.P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A.A. Ghorbani, CICIOT2023: a real-time dataset and benchmark for large-scale attacks in IoT environment, *Sensors* 23 (13) (2023) 5941.
- [45] A. Hamza, H.H. Gharakheili, T.A. Benson, V. Sivaraman, Detecting volumetric attacks on IoT devices via sdn-based monitoring of mud activity, in: Proceedings of the 2019 ACM Symposium on SDN Research, 2019, pp. 36–48.
- [46] B. Charyyev, M.H. Gunes, Locality-sensitive IoT network traffic fingerprinting for device identification, *IEEE Internet Things J.* 8 (3) (2020) 1272–1281.
- [47] B. Charyyev, M.H. Gunes, IoT traffic flow identification using locality sensitive hashes, in: ICC 2020-2020 IEEE International Conference on Communications (ICC), IEEE, 2020, pp. 1–6.
- [48] S. Dadkhah, H. Mahdikhani, P.K. Danso, A. Zohourian, K.A. Truong, A.A. Ghorbani, Towards the development of a realistic multidimensional IoT profiling dataset, 2022 19th Annu. Int. Conf. Privacy Secur. & Trust (PST) (2022). <https://www.unb.ca/cic/datasets/iotdataset-2022.html>. <https://doi.org/10.1109/PST55820.2022.9851966>
- [49] X.-S. Yang, Firefly algorithms for multimodal optimization, in: International Symposium on Stochastic Algorithms, Springer, 2009, pp. 169–178.
- [50] I. Fister, I. Fister, Jr, X.-S. Yang, J. Brest, A comprehensive review of firefly algorithms, *Swarm Evol. Comput.* 13 (2013) 34–46.
- [51] B. Selvakumar, K. Muneeswaran, Firefly algorithm based feature selection for network intrusion detection, *Comput. Secur.* 81 (2019) 148–155.
- [52] M. Karthikeyan, D. Manimegalai, K. RajaGopal, Firefly algorithm based WSN-IoT security enhancement with machine learning for intrusion detection, *Sci. Rep.* 14 (1) (2024) 231.
- [53] T.B. Ogunseyi, G. Thiagarajan, An explainable LSTM-based intrusion detection system optimized by firefly algorithm for IoT networks, *Sensors* 25 (7) (2025) 2288.
- [54] M.A. Pynadath, K.J. Pavithra, S.E. Lobo, S.S. Murthy, R. Bharathi, Anomaly detection and multi-output classification of IoT attacks, in: 2023 International Conference on Inventive Computation Technologies (ICICT), IEEE, 2023, pp. 1750–1757.
- [55] A. Zohourian, S. Dadkhah, H. Molyneaux, E.C.P. Neto, A.A. Ghorbani, IoT-PRIDS: leveraging packet representations for intrusion detection in IoT networks, *Comput. Secur.* 146 (2024) 104034.